A definitive guide to `mStats`

Version `3.1.0`

Myo Minn Oo

Updated on 2020-02-15

Best way to learn a new thing is to try it out.

Contents

1 Introduction **4**
1.1 About Author . . . 4
1.2 Reference . . . 4

2 Installing `mStats` package **5**
2.1 Simple way: install.packages() . . . 5
2.2 Using `devtools` package for advanced users . . . 5

3 Supporting functions **6**
3.1 Clean your workspace . . . 6
3.2 Logging your console outputs . . . 6

4 Functions for data management **8**
4.1 Viewing the dataset . . . 8
4.2 Describing the dataset or codebook . . . 8
4.3 Handling duplicated data . . . 9
4.4 Creating and transforming variables . . . 12
4.4.1 generate() . . . 12
4.4.2 egen() . . . 14
4.4.3 recode() . . . 15
4.5 Renaming variables . . . 17
4.6 Labels . . . 18
4.6.1 Variable labels . . . 18
4.6.2 Dataset labels . . . 19
4.6.3 Value labels . . . 21
4.7 Subsetting Data . . . 24
4.7.1 keep() or lose() . . . 24
4.7.2 pick() . . . 24
4.7.3 arrange() . . . 25
4.8 Formating Dates . . . 26
4.8.1 is.Date() . . . 26
4.8.2 day(), month(), year() . . . 26
4.8.3 formatDate() . . . 26

5 Functions for Data Analysis **28**
5.1 Epidemiological analysis . . . 28
5.1.1 Calculating Incidence Rates . . . 28
5.1.2 Diagnostic Test and ROC Curve . . . 32
5.2 Exploratory Data analysis . . . 38
5.2.1 One-way tabulation . . . 38
5.2.2 Cross-tabulation . . . 40
5.2.3 Number Summary . . . 42
5.2.4 Grouped Summary Measures . . . 43
5.3 Formatting model outputs . . . 46

6 Workflow in `mStats` package **50**
6.1 piping `%>%` from `magrittr` . . . 50
6.2 Milestone Project: Objectives . . . 50
6.3 Milestone Project: Code Demonstration . . . 51

7 Miscellaneous functions **67**
7.1 Duplicate observations . . . 67

1 Introduction

This R package can be described as **a tool for data management and Statistical Analysis**. It contains a set of functions aimed for epidemiologists and health researchers and includes three domains of commands.

1. Supporting Functions
2. Functions for data management
3. Functions for data analysis

1.1 About Author

I am Myo Minn Oo, currently a PhD Student and a dedicated R User. I have been a health researcher for 5+ years, starting out my initial journey mainly as `STATA` user and later switched to R. Hence, many of R functions in this package are inspired by my experiences and preferences in `STATA`. Read more about its history here.

This guide is not in-depth about explaining about arguments and every potential usage. It can be regarded as an overview of the package.

Any comments and suggestions are welcomed and you can reach me at email. Also visit me at my website.

1.2 Reference

- STATA DATA MANAGEMENT. UCLA: Statistical Consulting Group. https://stats.idre.ucla.edu/stata/seminars/stata-data-management/ (accessed February 13, 2020).
- LOGIT REGRESSION | R DATA ANALYSIS EXAMPLES. UCLA: Statistical Consulting Group. from https://stats.idre.ucla.edu/r/dae/logit-regression/ (accessed February 13, 2020).

2 Installing `mStats` package

It can be installed in two ways:

1. install.packages()
2. Using *devtools* package

2.1 Simple way: install.packages()

As of writing this guide, mStats version is `3.1.0`.

But Check the website for latest version.

```
install.packages("https://myominnoo.github.io/pages/R/r_archive/mStats_3.1.0.tar.gz",
                 repos = NULL, type = "source")
```

For any versions archived at my R Webpage, you can use corresponding `*.tar.gz` name.

```
install.packages("https://myominnoo.github.io/pages/R/r_archive/*.tar.gz",
                 repos = NULL, type = "source")
```

2.2 Using `devtools` package for advanced users

This lets you directly access to my `Github` repository and installs the latest version which may be a beta version. You also have no control over which version to be installed. If you would like to install stable version, please install as described in previous session.

Also visit `github` repository for source codes.

```
# installing devtools package
install.packages("devtools")
# direct installation from github repos
devtools::install_github("myominnoo/mStats")
```

3 Supporting functions

The purposes of these functions are for supporting when working with R and RStudio as well as for formatting outputs from other functions of data managment and analysis.

3.1 Clean your workspace

The foremost thing you should do when starting an R session is clear your workspace as R somtimes tends to save your previous work. You do not want your current project to be confused with your last ones.

The `clear()` function clean three places: Global Envionment, Console and Plots. It does not require any argument.

```
# loading the package
library(mStats)
# clean workspace
clear()
```

3.2 Logging your console outputs

You usually do this in R by using `sink()` function. It has two caveats.

1. You cannot both run and save the console outputs at the same time. It can be forcibly obtained using `split` argument.
2. Only outputs will be saved, but not commands. If you want to save commands, you can source the file by setting `echo = TRUE`. But there is also anther problem where your modelling commands sometimes produce error message, which stops the sourcing process.

To counter this and also inspired by STATA's `log` command, `ilog()` function comes in very handy. A log file is created by two processes: opening a text connection to our log file and closing it.

```
# Open the log file
ilog("~/R/myFirstLog.txt", append = FALSE)
```

```
##       log: C:/MEGA/GitHub/R/Dev_Folder/mStats_Guide/~/R/myFirstLog.txt
##   open on: 2020-02-15 12:14:26
##      note: replaced
## ----------------------------------------------------------------------
```

```
# load iris dataset
data(iris)
# Check data structure, its summary measures
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
summary(iris)
```

```
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
##  Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
##  1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
##  Median :5.800   Median :3.000   Median :4.350   Median :1.300
##  Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
##  3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
```

```
##  Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
##        Species
##  setosa    :50
##  versicolor:50
##  virginica :50
##
##
##
```

```
# Close log file
ilog.close()
```

4 Functions for data management

Data management, sometimes known as data processing, is an important aspect of data analysis workflow. Skillful data managment is essential to efficient workflow.

4.1 Viewing the dataset

R has built-in function `View()` for this purpose. If it is called, it opens a new spread-sheet like window. As a more convenient counterpart, `listView()` lists several variables of desire and display the values.

```
# listview variables of infert data
listView(infert, education:stratum)
```

```
##   education age parity induced case spontaneous stratum
## 1    0-5yrs  26      6       1    1           2       1
## 2    0-5yrs  42      1       1    1           0       2
## 3    0-5yrs  39      6       2    1           0       3
## 4    0-5yrs  34      4       2    1           0       4
## 5   6-11yrs  35      3       1    1           1       5
## 6   6-11yrs  36      4       2    1           1       6
```

```
# listview selected variables
listView(infert, c(education, case, stratum))
```

```
##   education case stratum
## 1    0-5yrs    1       1
## 2    0-5yrs    1       2
## 3    0-5yrs    1       3
## 4    0-5yrs    1       4
## 5   6-11yrs    1       5
## 6   6-11yrs    1       6
```

4.2 Describing the dataset or codebook

The `codebook` or its alias `describe()` provides the following information about the variables.

1. `VARS_NAME`: variable's name
2. `LABEL`: variable's label
3. `DATA_TYPE`: data type
4. `OBS_COUNT`: Number of Observations
5. `NA_COUNT`: Number of missing values (NA)
6. `NA_(%)`: Percentage of missing values (NA)

```
# codebook of iris dataset
codebook(iris)
```

```
## ========================================================
## Codebook: iris
##    label: NULL
## --------------------------------------------------------
##      VARS_NAME LABEL DATA_TYPE OBS_COUNT NA_COUNT NA_(%)
## 1 Sepal.Length  NULL   numeric       150        0    0 %
## 2  Sepal.Width  NULL   numeric       150        0    0 %
## 3 Petal.Length  NULL   numeric       150        0    0 %
## 4  Petal.Width  NULL   numeric       150        0    0 %
## 5      Species  NULL    factor       150        0    0 %
## ========================================================
```

4.3 Handling duplicated data

Finding duplicated records comes in very handy. Visually inspecting the dataset makes us almost impossible to spot duplications particularly if there is no unique ID for each records or the dataset is large.

The `duplicates()` function finds duplicated records by given variales or all variables (if such is not specified) and serially tags the records by creating a new variable `dupID_` intended for future uses. It produces a nice report of duplicated observations and surplus copies which is inspired by `STATA`'s `duplicates` command.

```
# finding duplicated records in iris dataset
iris_dup <- duplicates(iris)
```

```
## ======================================
## Duplicates in terms of all variables
## --------------------------------------
##   + Copies + Observations + Surplus +
## 1 |      1 |          148 |       0 |
## 2 |      2 |            2 |       1 |
## ======================================
## (Note:)
## (Total No. of obs: 150)
```

```
# check data structure
str(iris_dup)
```

```
## 'data.frame':    150 obs. of  6 variables:
##  $ Sepal.Length: num  4.3 4.4 4.4 4.4 4.5 4.6 4.6 4.6 4.6 4.7 ...
##  $ Sepal.Width : num  3 2.9 3 3.2 2.3 3.1 3.2 3.4 3.6 3.2 ...
##  $ Petal.Length: num  1.1 1.4 1.3 1.3 1.3 1.5 1.4 1.4 1 1.3 ...
##  $ Petal.Width : num  0.1 0.2 0.2 0.2 0.3 0.2 0.2 0.3 0.2 0.2 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ dupID_      : num  1 1 1 1 1 1 1 1 1 1 ...
```

Another advantage is that the function returns a data.frame so further processing can be pursued.

There are two more functions that either keep or remove only duplicated records.

- `keepUnique`: remove all duplicated records, making the dataset unique.
- `keepDup`: keep all duplicated records

```
# remove duplciated records
keepUnique(iris, Species)
```

```
## =======================================
## Duplicates in terms of Species
## ---------------------------------------
##    + Copies + Observations + Surplus +
## 1  |      1 |            0 |       0 |
## 2  |      2 |            0 |       0 |
## 3  |      3 |            0 |       0 |
## 4  |      4 |            0 |       0 |
## 5  |      5 |            0 |       0 |
## 6  |      6 |            0 |       0 |
## 7  |      7 |            0 |       0 |
## 8  |      8 |            0 |       0 |
## 9  |      9 |            0 |       0 |
## 10 |     10 |            0 |       0 |
## 11 |     11 |            0 |       0 |
## 12 |     12 |            0 |       0 |
```

```
## 13 |      13 |             0 |        0 |
## 14 |      14 |             0 |        0 |
## 15 |      15 |             0 |        0 |
## 16 |      16 |             0 |        0 |
## 17 |      17 |             0 |        0 |
## 18 |      18 |             0 |        0 |
## 19 |      19 |             0 |        0 |
## 20 |      20 |             0 |        0 |
## 21 |      21 |             0 |        0 |
## 22 |      22 |             0 |        0 |
## 23 |      23 |             0 |        0 |
## 24 |      24 |             0 |        0 |
## 25 |      25 |             0 |        0 |
## 26 |      26 |             0 |        0 |
## 27 |      27 |             0 |        0 |
## 28 |      28 |             0 |        0 |
## 29 |      29 |             0 |        0 |
## 30 |      30 |             0 |        0 |
## 31 |      31 |             0 |        0 |
## 32 |      32 |             0 |        0 |
## 33 |      33 |             0 |        0 |
## 34 |      34 |             0 |        0 |
## 35 |      35 |             0 |        0 |
## 36 |      36 |             0 |        0 |
## 37 |      37 |             0 |        0 |
## 38 |      38 |             0 |        0 |
## 39 |      39 |             0 |        0 |
## 40 |      40 |             0 |        0 |
## 41 |      41 |             0 |        0 |
## 42 |      42 |             0 |        0 |
## 43 |      43 |             0 |        0 |
## 44 |      44 |             0 |        0 |
## 45 |      45 |             0 |        0 |
## 46 |      46 |             0 |        0 |
## 47 |      47 |             0 |        0 |
## 48 |      48 |             0 |        0 |
## 49 |      49 |             0 |        0 |
## 50 |      50 |           150 |        3 |
## ========================================
## (Note:)
## (Total No. of obs: 150)
## (147 observations deleted)
```

```r
str(keepUnique(iris, Sepal.Length))
```

```
## ========================================
## Duplicates in terms of Sepal.Length
## ----------------------------------------
##    + Copies + Observations + Surplus +
## 1  |      1 |             9 |        0 |
## 2  |      2 |             4 |        2 |
## 3  |      3 |            12 |        4 |
## 4  |      4 |            20 |        5 |
## 5  |      5 |            10 |        2 |
## 6  |      6 |            30 |        5 |
```

```
## 7  |      7 |           21 |       3 |
## 8  |      8 |           16 |       2 |
## 9  |      9 |           18 |       2 |
## 10 |     10 |           10 |       1 |
## ========================================
## (Note:)
## (Total No. of obs: 150)
## (115 observations deleted)
## 'data.frame':    35 obs. of  5 variables:
##  $ Sepal.Length: num  4.3 4.4 4.5 4.6 4.7 4.8 4.9 5 5.1 5.2 ...
##  $ Sepal.Width : num  3 2.9 2.3 3.1 3.2 3.4 3 3.6 3.5 3.5 ...
##  $ Petal.Length: num  1.1 1.4 1.3 1.5 1.3 1.6 1.4 1.4 1.4 1.5 ...
##  $ Petal.Width : num  0.1 0.2 0.3 0.2 0.2 0.2 0.2 0.2 0.2 0.2 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```r
# keep duplicated records
str(keepDup(iris, Sepal.Length))
```

```
## ========================================
## Duplicates in terms of Sepal.Length
## ----------------------------------------
##    + Copies + Observations + Surplus +
## 1  |      1 |            9 |       0 |
## 2  |      2 |            4 |       2 |
## 3  |      3 |           12 |       4 |
## 4  |      4 |           20 |       5 |
## 5  |      5 |           10 |       2 |
## 6  |      6 |           30 |       5 |
## 7  |      7 |           21 |       3 |
## 8  |      8 |           16 |       2 |
## 9  |      9 |           18 |       2 |
## 10 |     10 |           10 |       1 |
## ========================================
## (Note:)
## (Total No. of obs: 150)
## (35 observations deleted)
## 'data.frame':    115 obs. of  5 variables:
##  $ Sepal.Length: num  4.4 4.4 4.6 4.6 4.6 4.7 4.8 4.8 4.8 4.8 ...
##  $ Sepal.Width : num  3 3.2 3.4 3.6 3.2 3.2 3 3.4 3.1 3 ...
##  $ Petal.Length: num  1.3 1.3 1.4 1 1.4 1.6 1.4 1.9 1.6 1.4 ...
##  $ Petal.Width : num  0.2 0.2 0.3 0.2 0.2 0.2 0.1 0.2 0.2 0.3 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

4.4 Creating and transforming variables

In most cases, the variables in the orginal datasets may not be what we need for data analysis. Hence, we either need to create new variables or edit existing ones by combining or transforming other variables.

4.4.1 generate()

This basic command `generate` can be used to create variables by performing arithmetic or logical operations on existing variables. It then returns a new data.frame containing the newly created variable.

Below we will create a new variable called `age.grp` from `age` of `infert` dataset. First, we will check its summary measures. Then, based on its mean value, we will split into two groups: one group where `age` is less than mean and the other one where it is greater than or equal to the mean.

Note the notification shown after running the `generate` command, indicating how many values have been created and the expression used to create this.

```
# check age of infert dataset
summary(infert$age)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   21.00   28.00   31.00   31.50   35.25   44.00
```

```
# generate age.grp based on mean value
infert_age <- generate(infert, age.grp, as.numeric(age < 31.50))
```

```
## (248 values generated)
## (Expression used: 'with(infert, as.numeric(age < 31.5))')
```

```
# check what's been created
str(infert_age)
```

```
## 'data.frame':    248 obs. of  9 variables:
##  $ education     : Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ age           : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ parity        : num  6 1 6 4 3 4 1 2 1 2 ...
##  $ induced       : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ case          : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ spontaneous   : num  2 0 0 0 1 1 0 0 1 0 ...
##  $ stratum       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ pooled.stratum: num  3 1 4 2 32 36 6 22 5 19 ...
##  $ age.grp       : num  1 0 0 0 0 0 1 0 1 1 ...
```

The example demonstrated above is quite easy and that's is not usually how we want our `age.grp`. Now, we will try to split age group into 5 groups: <= 25, 26-30, 31-35, 36-40, >40.

We can use this function with the help of `ifelse` function.

```
# create age.grp into 5 categories
infert_age5 <- generate(infert, age.grp,
         ifelse(age <= 25, 1,
                ifelse(age > 25 & age <= 30, 2,
                       ifelse(age > 30 & age <= 35, 3,
                              ifelse(age > 35 & age <= 40, 4,
                                     ifelse(age > 40, 5, NA))))))
```

```
## (248 values generated)
## (Expression used: 'with(infert, ifelse(age <= 25, 1, ifelse(age > 25 & age <= 30
## , 2, ifelse(age >     30 & age <= 35, 3, ifelse(age > 35 & age <= 40, 4, ifelse(
## age >     40, 5, NA))))))')
```

```r
# check what's been created
str(infert_age5)
```

```
## 'data.frame':    248 obs. of  9 variables:
##  $ education     : Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ age           : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ parity        : num  6 1 6 4 3 4 1 2 1 2 ...
##  $ induced       : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ case          : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ spontaneous   : num  2 0 0 0 1 1 0 0 1 0 ...
##  $ stratum       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ pooled.stratum: num  3 1 4 2 32 36 6 22 5 19 ...
##  $ age.grp       : num  2 5 4 3 3 4 1 3 1 2 ...
```

```r
# tabulate age.grp
tab(infert_age5$age.grp)
```

```
## ================================
## Tabulation: infert_age5$age.grp
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## 1        30     12.1         12.1
## 2        84     33.9         46.0
## 3        72     29.0         75.0
## 4        50     20.2         95.2
## 5        12      4.8        100.0
## Total   248    100.0        100.0
## ================================
```

4.4.2 egen()

The example shown above is somewhat tedious to type several `ifelse` statements. To ease it to a certain extent, we can use another function `egen` to cut down a numeric variable into a categorical one.

This `egen` function is an extended version of `generate` function where you can easily convert a dataset's variable into categorical one. You can specify the cut-points and corresponding labels as well as the name your variable will be created into. If you don't specify, the function will automatically do for you. Read the documentation for details.

Let's use the same example as above.

```
# generate age.grp by using egen
infert_egen <- egen(infert, age, cut = c(26, 31, 36, 41),
     lbl = c("<=25", "26-30", "31-35", "36-40", "41+"),
     new.var = age.grp)
```

```
## (248 values generated with labels: <=25, 26-30, 31-35, 36-40, 41+)
## ('age.grp' created & appended to dataframe 'infert')
```

```
# tabulate age.grp
tab(age.grp, infert_egen)
```

```
## ================================
## Tabulation: age.grp
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## <=25     30     12.1         12.1
## 26-30    84     33.9         46.0
## 31-35    72     29.0         75.0
## 36-40    50     20.2         95.2
## 41+      12      4.8        100.0
## Total   248    100.0        100.0
## ================================
```

This function also lets us know how many values has been generated.

The result from this tabulation output is the same as above, but the command is simpler. But the downside is that it can be used only for transforming numeric data to categorical ones.

4.4.3 recode()

Often variables in original datasets have to be coded into the values we want. This usually happens when there are too many categories or without meaningful values.

With the function `recode`, we can recode all values of a variable in one single step. From our `infert` dataset, we will recode `case` from 0 and 1 to 2 and 3.

```
# check case variable
tab(case, infert)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## 0       165     66.5         66.5
## 1        83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
```

```
# recoding 0 and 1 to 2 and 3
infert_recode <- recode(infert, case, c(0, 1), c(2, 3))
```

```
## (165 values recoded | '0' => '2')
## (83 values recoded | '1' => '3')
```

```
# check recoded case variable
tab(case, infert_recode)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## 2       165     66.5         66.5
## 3        83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
```

Here, the function also lets us know which values are coded into which ones.

Next, we will use the same variable but recode the numeric values into characters, from 0 and 1 to "No" and "Yes".

```
# recoding 0 and 1 to No and Yes
infert_recode <- recode(infert, case, c(0, 1), c("No", "Yes"))
```

```
## (165 values recoded | '0' => 'No')
## (83 values recoded | '1' => 'Yes')
```

```
# check recoded case variable
tab(case, infert_recode)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## No      165     66.5         66.5
```

```
## Yes      83     33.5        100.0
## Total   248    100.0        100.0
## ================================
```

4.5 Renaming variables

Renaming variables is a routine aspect of data management. With the function `rename`, variables' names can be changed by first specifying the old name and then the new name.

Let's rename `education` and `eduYears`.

```
# rename education to eduYears
str(rename(infert, education, eduYears))
```

```
## (Variable: renamed 'education' to 'eduYears')
## 'data.frame':    248 obs. of  8 variables:
##  $ eduYears      : Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ age           : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ parity        : num  6 1 6 4 3 4 1 2 1 2 ...
##  $ induced       : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ case          : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ spontaneous   : num  2 0 0 0 1 1 0 0 1 0 ...
##  $ stratum       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ pooled.stratum: num  3 1 4 2 32 36 6 22 5 19 ...
```

We can also rename multiple variables at the same times. Let's rename `age` to `AGE`, `parity` to `PARITY`, `induced` to `INDUCED`.

```
# rename multiple variables
str(rename(infert, c(age, parity, induced), c(AGE, PARITY, INDUCED)))
```

```
## (Variable: renamed 'age' to 'AGE')
## (Variable: renamed 'parity' to 'PARITY')
## (Variable: renamed 'induced' to 'INDUCED')
## 'data.frame':    248 obs. of  8 variables:
##  $ education     : Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ AGE           : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ PARITY        : num  6 1 6 4 3 4 1 2 1 2 ...
##  $ INDUCED       : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ case          : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ spontaneous   : num  2 0 0 0 1 1 0 0 1 0 ...
##  $ stratum       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ pooled.stratum: num  3 1 4 2 32 36 6 22 5 19 ...
```

4.6 Labels

Labels are usually attached to the corresponding variables to provide additional information about variables, either the meaning of the variable itself (variable label) or the meaning of the values of the variable (value labels).

4.6.1 Variable labels

Variable labels are used to expand the meaning of a variable beyond its name so that anyone can understand the meaning of variables used. We use `labelVars` to label variables.

In this function, `data` argument is optional, which means that you can label vectors.

Let's label `age` from `infert` dataset as `Age of patients`.

```
# label age
infert_ageLabel <- labelVars(infert, age, "age of patient")
```

```
## (Variable: labelled 'age' as 'age of patient')
```

```
# summarize age
summ(age, infert_ageLabel)
```

```
## ================================================================
## Number Summary: age
## label: age of patient
## ----------------------------------------------------------------
##     Obs. NA. Mean Std.Dev Median Q1   Q3 Min Max  CV. Normal.test
## age  248   0 31.5     5.3     31 28 35.2  21  44 16.7     0.00034
## ================================================================
```

Let's create a vector `x` and assign the label `dummy label`.

```
# create a vector x
x <- 1:100
# label x as dummy label
x <- labelVars(NULL, x, "dummy label")
```

```
## (Variable: labelled 'x' as 'dummy label')
```

```
# summarize x
summ(x)
```

```
## ================================================================
## Number Summary: x
## label: dummy label
## ----------------------------------------------------------------
##   Obs. NA. Mean Std.Dev Median   Q1   Q3 Min Max  CV. Normal.test
## x  100   0 50.5      29   50.5 25.8 75.2   1 100 57.4     0.00172
## ================================================================
```

This time we will label multiple variables at the same time.

```
# label multiple variables at the same time.
infert_multiLabel <- labelVars(
    infert,
    c(education, age, parity, induced, case),
    c("Patient's education in years", "Patient's age in Years",
      "Count of pregnancy", "Number of Induced abortions",
      "Case Status: 1 case and 0 control")
)
```

```
## (Variable: labelled 'education' as 'Patient's education in years')
## (Variable: labelled 'age' as 'Patient's age in Years')
## (Variable: labelled 'parity' as 'Count of pregnancy')
## (Variable: labelled 'induced' as 'Number of Induced abortions')
## (Variable: labelled 'case' as 'Case Status: 1 case and 0 control')
```

```r
# check the labels
codebook(infert_multiLabel)
```

```
## ==========================================================================================
## Codebook: infert_multiLabel
##    label: NULL
## ------------------------------------------------------------------------------------------
##        VARS_NAME                                LABEL DATA_TYPE OBS_COUNT NA_COUNT
## 1      education      Patient's education in years    factor       248        0
## 2            age            Patient's age in Years   numeric       248        0
## 3         parity                Count of pregnancy   numeric       248        0
## 4        induced       Number of Induced abortions   numeric       248        0
## 5           case Case Status: 1 case and 0 control   numeric       248        0
## 6    spontaneous                                NULL   numeric       248        0
## 7        stratum                                NULL   integer       248        0
## 8 pooled.stratum                                NULL   numeric       248        0
##   NA_(%)
## 1    0 %
## 2    0 %
## 3    0 %
## 4    0 %
## 5    0 %
## 6    0 %
## 7    0 %
## 8    0 %
## ==========================================================================================
```

4.6.2 Dataset labels

Datasets can be labelled so that it provides more meaningful information. It's also known as meta-data of the dataset.

Let's label the `infert` dataset.

```r
# Labelling the dataset
infert_label <- labelData(infert,
        "Infertility after Spontaneous and Induced Abortion")
```

```
## (Dataframe: labelled 'infert' as 'Infertility after Spontaneous and Induced Abor
## tion')
```

```r
# Check the label
codebook(infert_label)
```

```
## ==========================================================
## Codebook: infert_label
##    label: Infertility after Spontan
## eous and Induced Abortion
## ----------------------------------------------------------
##        VARS_NAME LABEL DATA_TYPE OBS_COUNT NA_COUNT NA_(%)
```

```
## 1      education  NULL    factor       248        0    0 %
## 2            age  NULL   numeric       248        0    0 %
## 3         parity  NULL   numeric       248        0    0 %
## 4        induced  NULL   numeric       248        0    0 %
## 5           case  NULL   numeric       248        0    0 %
## 6    spontaneous  NULL   numeric       248        0    0 %
## 7        stratum  NULL   integer       248        0    0 %
## 8 pooled.stratum  NULL   numeric       248        0    0 %
## ==========================================================
```

4.6.3 Value labels

Value labels give text descriptions to the numerical values of a variable. In R, this is equivalent to a factor variable. We can easily convert other data types into factors as well as changes their categories.

The order of the categories depend on the sequence appeared during tabulation.

```
# check tabulation sequence
tab(case, infert)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## 0       165     66.5         66.5
## 1        83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
```

```
# label values of case into no and yes
infert_case <- labelValues(infert, case, c("No", "Yes"))
```

```
## (Values: labelled in 'case' | '0' => 'No')
## (Values: labelled in 'case' | '1' => 'Yes')
```

```
# check tabulation again
tab(case, infert_case)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## No      165     66.5         66.5
## Yes      83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
```

Similar to previous functions, it also allows value labelling of multiple variables.

Let's label case as No and Yes, induced as none, one and two, education as low, mide and high, at the same time.

```
# check tabulation sequence
tab(c(case, induced, education), infert)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ----------------------------------
##       Freq. Percent. Cum.Percent.
## 0       165     66.5         66.5
## 1        83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
## ==================================
## Tabulation: induced
## label: NULL
```

```
## ---------------------------------
##       Freq. Percent. Cum.Percent.
## 0       143     57.7         57.7
## 1        68     27.4         85.1
## 2        37     14.9        100.0
## Total   248    100.0        100.0
## ==================================
## ====================================
## Tabulation: education
## label: NULL
## ------------------------------------
##         Freq. Percent. Cum.Percent.
## 0-5yrs     12      4.8          4.8
## 6-11yrs   120     48.4         53.2
## 12+ yrs   116     46.8        100.0
## Total     248    100.0        100.0
## ====================================
```

```r
# label them at the same time
infert_new <- labelValues(
    infert,
    list(case, induced, education),
    list(c("No", "Yes"),
         c("None", "One", "Two"),
         c("Low", "Mid", "High"))
)
```

```
## (Values: labelled in 'case' | '0' => 'No')
## (Values: labelled in 'case' | '1' => 'Yes')
## (Values: labelled in 'induced' | '0' => 'None')
## (Values: labelled in 'induced' | '1' => 'One')
## (Values: labelled in 'induced' | '2' => 'Two')
## (Values: labelled in 'education' | '0-5yrs' => 'Low')
## (Values: labelled in 'education' | '6-11yrs' => 'Mid')
## (Values: labelled in 'education' | '12+ yrs' => 'High')
```

```r
# check tab again
tab(c(case, induced, education), infert_new)
```

```
## ==================================
## Tabulation: case
## label: NULL
## ---------------------------------
##       Freq. Percent. Cum.Percent.
## No      165     66.5         66.5
## Yes      83     33.5        100.0
## Total   248    100.0        100.0
## ==================================
## ==================================
## Tabulation: induced
## label: NULL
## ---------------------------------
##       Freq. Percent. Cum.Percent.
## None    143     57.7         57.7
## One      68     27.4         85.1
## Two      37     14.9        100.0
```

```
## Total   248    100.0        100.0
## ================================
## ================================
## Tabulation: education
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## Low      12      4.8          4.8
## Mid     120     48.4         53.2
## High    116     46.8        100.0
## Total   248    100.0        100.0
## ================================
```

4.7 Subsetting Data

Often your original dataset is big with hundreds of variables and tens of thousands of observations. In such cases, you can `keep` or `lose` **variables** of interest as well as `pick` **observations** of interest, in order to make the dataset small enough for you to manage.

These functions return the changed dataset that can be assigned for further processing. They also display the notification about changes being made, which is a nice feature of this package.

4.7.1 keep() or lose()

Sometimes you do not want all of the variables in a data file. You can use the `keep` and `lose` functions to subset variables.

If we think of your data like a spreadsheet, this will show how you can remove variables from your data. Let's try this with the `infert` dataset.

```
# keep three variables: education, age and parity
str(keep(infert, education, age, parity))
```

```
## (3 variables kept: education | age | parity)
## 'data.frame':    248 obs. of  3 variables:
##  $ education: Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ age      : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ parity   : num  6 1 6 4 3 4 1 2 1 2 ...
```

```
# lose education and case
str(lose(infert, education, case, parity, stratum, pooled.stratum))
```

```
## (5 variables deleted: education | case | parity | stratum | pooled.stratum)
## 'data.frame':    248 obs. of  3 variables:
##  $ age        : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ induced    : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ spontaneous: num  2 0 0 0 1 1 0 0 1 0 ...
```

These showed how to use `keep` and `lose` variables to remove variables from your dataset.

4.7.2 pick()

Next, the function `pick` can be used to keep and drop observations. Thinking of your data like a spreadsheet, `pick` can be used to remove rows of your data.

```
# we want age between 30 and 40.
infert_age <- pick(infert, age >= 30 & age <= 40)
```

```
## (134 observations picked)
## (Expression used: 'infert[ with(infert, age >= 30 & age <= 40), ]')
```

```
# check the summary
summ(age, infert_age)
```

```
## ==============================================================
## Number Summary: age
## label: NULL
## --------------------------------------------------------------
##     Obs. NA. Mean Std.Dev Median Q1 Q3 Min Max CV. Normal.test
## age  134   0 34.4       3     35 32 37  30  40 8.6     0.00001
## ==============================================================
```

4.7.3 arrange()

This function arranges the order of observations based on the combination of several variables. Putting negative sign - in front of the variables will arrange observations in descending order and if nothing is specified, it will be in ascending order.

```
# arrange by age in ascending order
head(arrange(infert, age))
```

```
## (dataframe arranged by: 'age')
## (Expression used: 'infert[ with(infert, order(age) ), ]')

##     education age parity induced case spontaneous stratum pooled.stratum
## 9     6-11yrs  21      1       0    1           1       9              5
## 67    12+ yrs  21      1       0    1           1      67             39
## 92    6-11yrs  21      1       0    0           1       9              5
## 150   12+ yrs  21      1       0    0           1      67             39
## 174   6-11yrs  21      1       1    0           0       9              5
## 232   12+ yrs  21      1       0    0           0      67             39
```

```
# by descending order
head(arrange(infert, -age))
```

```
## (dataframe arranged by: '-age')
## (Expression used: 'infert[ with(infert, order(-age) ), ]')

##     education age parity induced case spontaneous stratum pooled.stratum
## 20    6-11yrs  44      1       0    1           1      20             17
## 103   6-11yrs  44      1       0    0           0      20             17
## 185   6-11yrs  44      1       1    0           0      20             17
## 2      0-5yrs  42      1       1    1           0       2              1
## 33    6-11yrs  42      1       1    1           0      33             16
## 85     0-5yrs  42      1       0    0           0       2              1
```

```
# mixed order of several variables
head(arrange(infert, age, education, -case))
```

```
## (dataframe arranged by: 'age' => 'education' => '-case')
## (Expression used: 'infert[ with(infert, order(age, education, -case) ), ]')

##     education age parity induced case spontaneous stratum pooled.stratum
## 9     6-11yrs  21      1       0    1           1       9              5
## 92    6-11yrs  21      1       0    0           1       9              5
## 174   6-11yrs  21      1       1    0           0       9              5
## 67    12+ yrs  21      1       0    1           1      67             39
## 150   12+ yrs  21      1       0    0           1      67             39
## 232   12+ yrs  21      1       0    0           0      67             39
```

Also note the notification.

4.8 Formating Dates

This set of functions deals with `Date` format in R.

4.8.1 is.Date()

As very simple as it is, the function to check whether an object is of `Date` data type is missing in base R.

```
# Creating a Date vector
x <- as.Date(c("2020-01-01", "2020-01-02", "2020-01-03",
               "2020-01-04", "2020-01-05"), format = "%Y-%m-%d")
# check its type
typeof(x)
```

```
## [1] "double"
```

```
# check whether it is a Date format
is.Date(x)
```

```
## [1] TRUE
```

4.8.2 day(), month(), year()

Similar to the previous one, base R does not provide functions to extract days or months or years from Dates.

```
# Creating a Date vector
x <- as.Date(c("2020-01-01", "2020-01-02", "2020-01-03",
               "2020-01-04", "2020-01-05"), format = "%Y-%m-%d")
# extract years
year(x)
```

```
## [1] 2020 2020 2020 2020 2020
```

```
# extract months
month(x)
```

```
## [1] 1 1 1 1 1
```

```
# extract days
day(x)
```

```
## [1] 1 2 3 4 5
```

4.8.3 formatDate()

Often Date variables are read from original dataset as either characters or numbers. In this case, `formatDate` can quickly convert these into Dates.

First, let's convert some characters into Dates.

```
# creating a character vector
y <- c("2019-01-15", "2019-01-20", "2019-01-21", "2019-01-22")
# check if y is a Date
is.Date(y)
```

```
## [1] FALSE
```

```
# change to Dates
y.dates <- formatDate(y, "Ymd", "-")
# check if y.dates is a Date
is.Date(y.dates)
```

```
## [1] TRUE
```

Next, we will convert numbers to Dates.

```r
# creating numeric vector
y <- 42705: 42805
# convert to Dates
y.dates <- formatDate(y)
y.dates
```

```
##   [1] "2016-12-01" "2016-12-02" "2016-12-03" "2016-12-04" "2016-12-05"
##   [6] "2016-12-06" "2016-12-07" "2016-12-08" "2016-12-09" "2016-12-10"
##  [11] "2016-12-11" "2016-12-12" "2016-12-13" "2016-12-14" "2016-12-15"
##  [16] "2016-12-16" "2016-12-17" "2016-12-18" "2016-12-19" "2016-12-20"
##  [21] "2016-12-21" "2016-12-22" "2016-12-23" "2016-12-24" "2016-12-25"
##  [26] "2016-12-26" "2016-12-27" "2016-12-28" "2016-12-29" "2016-12-30"
##  [31] "2016-12-31" "2017-01-01" "2017-01-02" "2017-01-03" "2017-01-04"
##  [36] "2017-01-05" "2017-01-06" "2017-01-07" "2017-01-08" "2017-01-09"
##  [41] "2017-01-10" "2017-01-11" "2017-01-12" "2017-01-13" "2017-01-14"
##  [46] "2017-01-15" "2017-01-16" "2017-01-17" "2017-01-18" "2017-01-19"
##  [51] "2017-01-20" "2017-01-21" "2017-01-22" "2017-01-23" "2017-01-24"
##  [56] "2017-01-25" "2017-01-26" "2017-01-27" "2017-01-28" "2017-01-29"
##  [61] "2017-01-30" "2017-01-31" "2017-02-01" "2017-02-02" "2017-02-03"
##  [66] "2017-02-04" "2017-02-05" "2017-02-06" "2017-02-07" "2017-02-08"
##  [71] "2017-02-09" "2017-02-10" "2017-02-11" "2017-02-12" "2017-02-13"
##  [76] "2017-02-14" "2017-02-15" "2017-02-16" "2017-02-17" "2017-02-18"
##  [81] "2017-02-19" "2017-02-20" "2017-02-21" "2017-02-22" "2017-02-23"
##  [86] "2017-02-24" "2017-02-25" "2017-02-26" "2017-02-27" "2017-02-28"
##  [91] "2017-03-01" "2017-03-02" "2017-03-03" "2017-03-04" "2017-03-05"
##  [96] "2017-03-06" "2017-03-07" "2017-03-08" "2017-03-09" "2017-03-10"
## [101] "2017-03-11"
```

```r
# check if y.dates is Date
is.Date(y.dates)
```

```
## [1] TRUE
```

5 Functions for Data Analysis

This domain is regarded with activities taken after getting the final clean data. It can be further separated into three sets of functions.

1. Epidemiological analysis
2. Exploratory data analysis
3. Formatting outputs of Modelling

5.1 Epidemiological analysis

This set of functions are intended to help in epidemiological calculations.

5.1.1 Calculating Incidence Rates

Rates of event occurrences, known as incidence rates are outcome measures in longitudinal studies. In most longitudinal studies, follow-up times vary due to logistic resasons, different periods of recruitment, delay enrolment into the study, lost-to-follow-up, immigration or emigration and death.

Follow-up time in longitudinal studies

Period of observation (called as follow-up time) starts when individuals join the study and ends when they either have an outcome of interest, are lost-to- follow-up or the follow-up period ends, whichever happens first. This period is called person-year-at-risk. This is denoted by PY in strate function's output and numer of event by D.

The function `strate` calculates Incidence Rates per population as well as its 95% confidence intervals. The calculation can also be stratified by another variable.

The following demonstrates two examples of calculating stratified incidence rates: by sex and age groups.

In the first example, sex is recoded into male and female and then the analysis is performed.

In the second example, age groups are created by uisng `egen` function based on 51, 61, and 71 cut-off points. Then incidence rates are calculated.

```r
# load survival library to access lung data
library(survival)
data(lung)
# check data structure
str(lung)
```

```
## 'data.frame':    228 obs. of  10 variables:
##  $ inst     : num  3 3 3 5 1 12 7 11 1 7 ...
##  $ time     : num  306 455 1010 210 883 ...
##  $ status   : num  2 2 1 2 2 1 2 2 2 2 ...
##  $ age      : num  74 68 56 57 60 74 68 71 53 61 ...
##  $ sex      : num  1 1 1 1 1 1 2 2 1 1 ...
##  $ ph.ecog  : num  1 0 0 1 0 1 2 2 1 2 ...
##  $ ph.karno : num  90 90 90 90 100 50 70 60 70 70 ...
##  $ pat.karno: num  100 90 90 60 90 80 60 80 80 70 ...
##  $ meal.cal : num  1175 1225 NA 1150 NA ...
##  $ wt.loss  : num  NA 15 15 11 0 0 10 1 16 34 ...
```

```r
# codebook the data
codebook(lung)
```

```
## ========================================================
## Codebook: lung
##    label: NULL
```

```
## ------------------------------------------------------
##    VARS_NAME LABEL DATA_TYPE OBS_COUNT NA_COUNT NA_(%)
## 1       inst  NULL   numeric       227        1  0.4 %
## 2       time  NULL   numeric       228        0    0 %
## 3     status  NULL   numeric       228        0    0 %
## 4        age  NULL   numeric       228        0    0 %
## 5        sex  NULL   numeric       228        0    0 %
## 6    ph.ecog  NULL   numeric       227        1  0.4 %
## 7   ph.karno  NULL   numeric       227        1  0.4 %
## 8  pat.karno  NULL   numeric       225        3  1.3 %
## 9   meal.cal  NULL   numeric       181       47 20.6 %
## 10   wt.loss  NULL   numeric       214       14  6.1 %
## ======================================================
```

```r
## incidence rates
strate(lung, time, status, per = 100)
```

```
## ====================================================
## Tabulation of failure rates
## failure: status == 1
## ----------------------------------------------------
##   Event Person Year Inc Rate (IR) Lower CI Upper CI
## 1    63      695.93       0.09053  0.07072  0.11588
## ====================================================
## (Notes:)
## (Estimated per 100person-years-at-risk)
## (228 records included in the analysis)
```

```r
# faliure event can be specified
strate(lung, time, status, fail = 2, per = 1000)
```

```
## ====================================================
## Tabulation of failure rates
## failure: status == 2
## ----------------------------------------------------
##   Event Person Year Inc Rate (IR) Lower CI Upper CI
## 1   165      69.593       2.37093  2.03541  2.76176
## ====================================================
## (Notes:)
## (Estimated per 1000person-years-at-risk)
## (228 records included in the analysis)
```

```r
# recoding sex
lung <- recode(lung, sex, c(1, 2), c("Male", "Female"))
```

```
## (138 values recoded | '1' => 'Male')
## (90 values recoded | '2' => 'Female')
```

```r
# check cross-tabulation
xtab(sex, status, lung)
```

```
## =====================================================
## Tabulation: sex ~ status
## -----------------------------------------------------
##         1 (r%)   2 (r%) Total (r%) Chi.Square F.Exact
## Female 37 41.1  53 58.9    90  100    0.00024 0.00043
## Male   26 18.8 112 81.2   138  100
```

```
## Total  63 27.6 165 72.4   228  100
## ======================================================
```

```r
## incidence rates stratified by sex
strate(lung, time, status, by = sex, per = 1000)
```

```
## ==========================================================
## Tabulation of failure rates
## failure: status == 1
## by categories of: sex
## ----------------------------------------------------------
##        Event Person Year Inc Rate (IR) Lower CI Upper CI
## Female    37      30.507       1.21284  0.87875  1.67395
## Male      26      39.086       0.66520  0.45291  0.97699
## ==========================================================
## (Notes:)
## (categories of sex: Female | Male)
## (Estimated per 1000person-years-at-risk)
## (228 records included in the analysis)
```

```r
# summarize age
summ(age, lung)
```

```
## ================================================================
## Number Summary: age
## label: NULL
## ----------------------------------------------------------------
##     Obs. NA. Mean Std.Dev Median Q1 Q3 Min Max  CV. Normal.test
## age  228   0 62.4     9.1     63 56 69  39  82 14.5     0.00483
## ================================================================
```

```r
# getting age.grp into 4 groups
lung <- egen(lung, age, c(51, 61, 71),
             c("<=50", "51-60", "61-70", "71+"), age.grp)
```

```
## (228 values generated with labels: <=50, 51-60, 61-70, 71+)
## ('age.grp' created & appended to dataframe 'lung')
```

```r
# check cross-tabulation
xtab(age.grp, status, lung)
```

```
## ======================================================
## Tabulation: age.grp ~ status
## ------------------------------------------------------
##        1 (r%)   2 (r%) Total (r%) Chi.Square F.Exact
## <=50  10 38.5  16 61.5    26  100    0.25383 0.24022
## 51-60 20 29.4  48 70.6    68  100
## 61-70 25 28.4  63 71.6    88  100
## 71+    8 17.4  38 82.6    46  100
## Total 63 27.6 165 72.4   228  100
## ======================================================
```

```r
## incidence rates stratified by age.grp
strate(lung, time, status, by = age.grp, per = 1000)
```

```
## ==========================================================
## Tabulation of failure rates
## failure: status == 1
```

```
## by categories of: age.grp
## ------------------------------------------------------
##       Event Person Year Inc Rate (IR) Lower CI Upper CI
## <=50     10       8.494       1.17730  0.63345  2.18810
## 51-60    20      22.036       0.90761  0.58554  1.40681
## 61-70    25      26.963       0.92720  0.62651  1.37219
## 71+       8        12.1       0.66116  0.33064  1.32207
## ======================================================
## (Notes:)
## (categories of age.grp: <=50 | 51-60 | 61-70 | 71+)
## (Estimated per 1000person-years-at-risk)
## (228 records included in the analysis)
```

5.1.2 Diagnostic Test and ROC Curve

The screening tests are based on Bayes' Theorem. These tests help clinicians to correctly predict the presence or absence of a particular disease from the knowlege of test results (positive or negative) and/or the status of presenting symptoms (present or absent) or information regarding the likelihood of positive and negative test results and the likelihood of the presence or absence of a particular symptom in patients with and without a particular disease.

Statistics of diagnostic tests:

1. Sensitivity: True Positive (TP) rate among diseased (D+) = TP / D+
2. Specificity: True Negative (TN) rate among non-diseased (D-) = TN / D-
3. Positive predictive value (PPV): probability of D+ when all positives (AP) = D+ / AP
4. Negative predictive value (NPV): probability of D- when all negatives (AN) = D+ / AN
5. Likelihood ratio (LR) = To summarize information about a diagnostic test LR of positive result (LR+) = (Sensitivity) / (1 - Specificity) LR of negative result (LR-) = (1 - Sensitivity) / (Specificity)

Calculating confidence intervals:

The method was developed by Edwin Bidwell Wilson in 1927. This interval has good properties even for a small sample. Just lik Pearson's chi-squared test and Yates' continuity correction.

Several other methods have been developed such as Clopper–Pearson interval and Agresti–Coull interval. But Wilson score interval methods (with or without continuity correction) have been shown to be the most accurate and the most robust. For further details, see documentations in R.

The function `diagTest` reports statistics of diagnostic tests. It allows 2x2 matrix as well as dataset as inputs.

Let's try two examples. They comes from the following book and journal:

1. Biostatistics A Foundation for Analysis in the Health Sciences (10th Edition)
2. Avijit Hazra, Nithya Gogtay. Biostatistics Series Module 7: The Statistics of Diagnostic Tests. Indian J Dermatol. 2017 Jan-Feb; 62(1): 18-24. doi: 10.4103/0019-5154.198047

```r
## Example from Biostatistics Textbook
# creating a 2x2 table
x <- as.table(matrix(c(436, 5, 14, 495), ncol = 2, byrow = TRUE))
# running diagTest
diagTest(x)
```

```
## ========================================================================
## Cross-Tabulation of Screening Test and Disease Status
##
##                Disease
## Test            Present (+) Absent (-) Total
##   Positive (+)         436          5   441
##   Negative (-)          14        495   509
##   Total                450        500   950
##
##     Sample Prevalence: 47.37 %
## Population Prevalence: NOT GIVEN
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##          Statistics of Diagnostic Tests
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        96.89       (94.85 - 98.14)           (94.75 - 98.01)
## Specificity        99.00       (97.68 - 99.57)           (97.60 - 99.45)
## PPV                98.87       (97.37 - 99.51)           (97.28 - 99.38)
## NPV                97.25       (95.44 - 98.35)           (95.35 - 98.24)
```

```
## LR (+)             96.89
## LR (-)              0.03
## PTP (LR+)          98.87
## PTP (LR-)           2.75
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
## (Note: used 'Sample' prevalence in the calculation of PPV and NPV.)
## (PPV or NPV: Positive or Negative Predictive Value)
## (LR (+) or (-): Likelihood Ratio of a positive or negative result)
## (PTP (LR+): Post-Test Probability of disease given (+) test)
## (PTP (LR-): Post-Test Probability of non-disease given (-) test)
## ====================================================================
```

```r
# Given population prevalence
diagTest(x, p = .113)
```

```
## ====================================================================
## Cross-Tabulation of Screening Test and Disease Status
##
##                Disease
## Test            Present (+) Absent (-) Total
##   Positive (+)         436          5   441
##   Negative (-)          14        495   509
##   Total                450        500   950
##
##     Sample Prevalence: 47.37 %
## Population Prevalence: 11.30 %
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##          Statistics of Diagnostic Tests
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        96.89       (94.85 - 98.14)           (94.75 - 98.01)
## Specificity        99.00       (97.68 - 99.57)           (97.60 - 99.45)
## PPV                92.51       (89.66 - 94.61)           (89.56 - 94.49)
## NPV                99.60       (98.57 - 99.89)           (98.50 - 99.76)
## LR (+)             96.89
## LR (-)              0.03
## PTP (LR+)          92.51
## PTP (LR-)           0.40
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
## (Note: used 'Population' prevalence in the calculation of PPV and NPV.)
## (PPV or NPV: Positive or Negative Predictive Value)
## (LR (+) or (-): Likelihood Ratio of a positive or negative result)
## (PTP (LR+): Post-Test Probability of disease given (+) test)
## (PTP (LR-): Post-Test Probability of non-disease given (-) test)
## ====================================================================
```

```r
## Example from a journal paper
y <- as.table(matrix(c(2, 18, 1, 182), ncol = 2, byrow = TRUE))
diagTest(y)
```

```
## ====================================================================
## Cross-Tabulation of Screening Test and Disease Status
##
##                Disease
## Test            Present (+) Absent (-) Total
##   Positive (+)           2         18    20
```

```
##   Negative (-)           1        182   183
##   Total                  3        200   203
##
##     Sample Prevalence: 1.48 %
## Population Prevalence: NOT GIVEN
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##          Statistics of Diagnostic Tests
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        66.67       (20.77 - 93.85)           (17.41 - 82.59)
## Specificity        91.00       (86.22 - 94.23)           (86.01 - 93.96)
## PPV                10.00        (2.79 - 30.10)            (1.36 - 27.34)
## NPV                99.45       (96.97 - 99.90)           (96.80 - 99.54)
## LR (+)              7.41
## LR (-)              0.37
## PTP (LR+)          10.00
## PTP (LR-)           0.55
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
## (Note: used 'Sample' prevalence in the calculation of PPV and NPV.)
## (PPV or NPV: Positive or Negative Predictive Value)
## (LR (+) or (-): Likelihood Ratio of a positive or negative result)
## (PTP (LR+): Post-Test Probability of disease given (+) test)
## (PTP (LR-): Post-Test Probability of non-disease given (-) test)
## ========================================================================
```

```r
diagTest(y, p = .10)
```

```
## ========================================================================
## Cross-Tabulation of Screening Test and Disease Status
##
##               Disease
## Test           Present (+) Absent (-) Total
##   Positive (+)           2         18    20
##   Negative (-)           1        182   183
##   Total                  3        200   203
##
##     Sample Prevalence: 1.48 %
## Population Prevalence: 10.00 %
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##          Statistics of Diagnostic Tests
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        66.67       (20.77 - 93.85)           (17.41 - 82.59)
## Specificity        91.00       (86.22 - 94.23)           (86.01 - 93.96)
## PPV                45.15       (25.94 - 65.92)           (24.29 - 63.37)
## NPV                96.09       (92.20 - 98.08)           (91.98 - 97.76)
## LR (+)              7.41
## LR (-)              0.37
## PTP (LR+)          45.15
## PTP (LR-)           3.91
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
## (Note: used 'Population' prevalence in the calculation of PPV and NPV.)
## (PPV or NPV: Positive or Negative Predictive Value)
## (LR (+) or (-): Likelihood Ratio of a positive or negative result)
## (PTP (LR+): Post-Test Probability of disease given (+) test)
```

```
## (PTP (LR-): Post-Test Probability of non-disease given (-) test)
## ========================================================================
```

Drawing ROC Curves

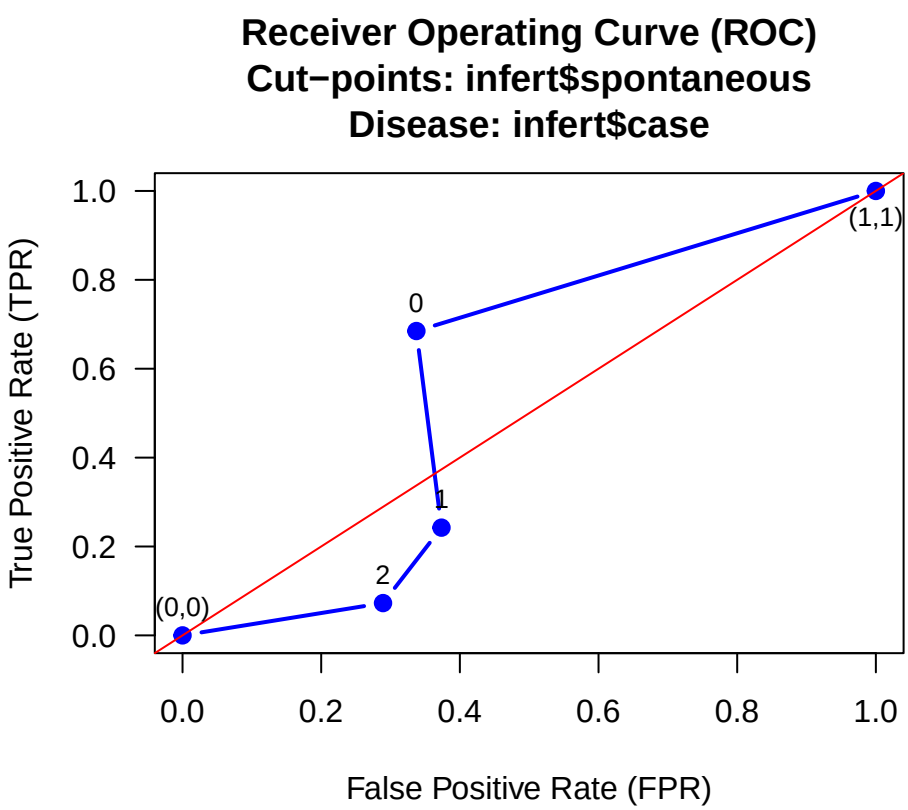
A receiver operating characteristic curve, or ROC curve, is a graphical plot that illustrates the diagnostic ability of a binary data.

The ROC curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings. This mean that a comparison of two operating characteristics (TPR and FPR) has been made as the criterion changes.

```r
# tab case and spontaneous of infert
xtab(spontaneous, case, infert)
```

```
## ====================================================
## Tabulation: spontaneous ~ case
## ----------------------------------------------------
##          0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## 0      113 80.1 28 19.9   141  100    0.00000 0.00000
## 1       40 56.3 31 43.7    71  100
## 2       12 33.3 24 66.7    36  100
## Total 165 66.5 83 33.5   248  100
## ====================================================
```

```r
## Diag Test as well as ROC curve
diagTest(infert$case, infert$spontaneous)
```




```
## ========================================================================
```

```
## Cross-Tabulations of infert$spontaneous and infert$case
##
##                    infert$case
## infert$spontaneous   0  1 Total
##              0     113 28   141
##              1      40 31    71
##              2      12 24    36
##              Total 165 83   248
##
##     Sample Prevalence: 72.17 %
## Population Prevalence: NOT GIVEN
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##       Different Cut-off points and Statistics of Diagnostic Tests
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
##                    infert$case
## infert$spontaneous   0  1
##                0   113 28
##                1|2  52 55
##  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        68.48       (61.04 - 75.08)           (60.77 - 74.76)
## Specificity        66.27       (55.58 - 75.52)           (55.06 - 74.88)
## PPV                84.04       (77.10 - 89.17)           (76.79 - 88.78)
## NPV                44.78       (35.70 - 54.22)           (35.29 - 53.72)
## LR (+)              2.03
## LR (-)              0.48
## PTP (LR+)          84.04
## PTP (LR-)          55.22
## ----------------------------------------------------------------------
##                    infert$case
## infert$spontaneous   0  1
##                1    40 31
##                0|2 125 52
##  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity        24.24       (18.34 - 31.32)           (18.07 - 31.00)
## Specificity        62.65       (51.90 - 72.28)           (51.39 - 71.64)
## PPV                62.73       (51.10 - 73.05)           (50.51 - 72.30)
## NPV                24.18       (18.46 - 30.99)           (18.21 - 30.69)
## LR (+)              0.65
## LR (-)              1.21
## PTP (LR+)          62.73
## PTP (LR-)          75.82
## ----------------------------------------------------------------------
##                    infert$case
## infert$spontaneous   0  1
##                2    12 24
##                0|1 153 59
##  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
##             Estimate (%) 95% CI (Wilson Score) 95% CI (Score Correction)
## Sensitivity         7.27        (4.21 - 12.28)            (3.95 - 11.94)
## Specificity        71.08       (60.57 - 79.73)           (60.06 - 79.09)
## PPV                39.48       (25.28 - 55.70)           (24.23 - 54.24)
## NPV                22.82       (17.68 - 28.92)           (17.47 - 28.67)
```

```
## LR (+)              0.25
## LR (-)              1.30
## PTP (LR+)          39.48
## PTP (LR-)          77.18
## ---------------------------------------------------------------------
## ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
## (Note: used 'Sample' prevalence in the calculation of PPV and NPV.)
## (PPV or NPV: Positive or Negative Predictive Value)
## (LR (+) or (-): Likelihood Ratio of a positive or negative result)
## (PTP (LR+): Post-Test Probability of disease given (+) test)
## (PTP (LR-): Post-Test Probability of non-disease given (-) test)
## =====================================================================
```

5.2 Exploratory Data analysis

After data mangement, exploring data is all about understanding your data. Sometimes this is called descriptive or summary measures. That means it depends on the nature and type of the data.

5.2.1 One-way tabulation

Frequency distribution is the most common summary measure for categorical data, either nominal or ordinal data. With `tab`, tabulation of either one variable, several variables or the whole dataset becomes very easy.

Let's find the frequency distribution of variables from `esoph` dataset.

```
# data esoph
str(esoph)
```

```
## 'data.frame':    88 obs. of  5 variables:
##  $ agegp    : Ord.factor w/ 6 levels "25-34"<"35-44"<..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ alcgp    : Ord.factor w/ 4 levels "0-39g/day"<"40-79"<..: 1 1 1 1 2 2 2 2 3 3 ...
##  $ tobgp    : Ord.factor w/ 4 levels "0-9g/day"<"10-19"<..: 1 2 3 4 1 2 3 4 1 2 ...
##  $ ncases   : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ ncontrols: num  40 10 6 5 27 7 4 7 2 1 ...
```

```
# tabulation of agegp
tab(agegp, esoph)
```

```
## ================================
## Tabulation: agegp
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## 25-34    15     17.0         17.0
## 35-44    15     17.0         34.1
## 45-54    16     18.2         52.3
## 55-64    16     18.2         70.5
## 65-74    15     17.0         87.5
## 75+      11     12.5        100.0
## Total    88    100.0        100.0
## ================================
```

```
# one-way tabulation of agegp, alcgph, tobgp
tab(c(agegp, alcgp, tobgp), esoph)
```

```
## ================================
## Tabulation: agegp
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## 25-34    15     17.0         17.0
## 35-44    15     17.0         34.1
## 45-54    16     18.2         52.3
## 55-64    16     18.2         70.5
## 65-74    15     17.0         87.5
## 75+      11     12.5        100.0
## Total    88    100.0        100.0
## ================================
## ====================================
## Tabulation: alcgp
## label: NULL
```

```
## ------------------------------------
##           Freq. Percent. Cum.Percent.
## 0-39g/day    23     26.1         26.1
## 40-79        23     26.1         52.3
## 80-119       21     23.9         76.1
## 120+         21     23.9        100.0
## Total        88    100.0        100.0
## ====================================
## ===================================
## Tabulation: tobgp
## label: NULL
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## 0-9g/day    24     27.3         27.3
## 10-19       24     27.3         54.5
## 20-29       20     22.7         77.3
## 30+         20     22.7        100.0
## Total       88    100.0        100.0
## ===================================
```

```r
# one-way tab of esoph
tab(esoph)
```

```
## ================================
## Tabulation: agegp
## label: NULL
## --------------------------------
##       Freq. Percent. Cum.Percent.
## 25-34    15     17.0         17.0
## 35-44    15     17.0         34.1
## 45-54    16     18.2         52.3
## 55-64    16     18.2         70.5
## 65-74    15     17.0         87.5
## 75+      11     12.5        100.0
## Total    88    100.0        100.0
## ================================
## ====================================
## Tabulation: alcgp
## label: NULL
## ------------------------------------
##           Freq. Percent. Cum.Percent.
## 0-39g/day    23     26.1         26.1
## 40-79        23     26.1         52.3
## 80-119       21     23.9         76.1
## 120+         21     23.9        100.0
## Total        88    100.0        100.0
## ====================================
## ===================================
## Tabulation: tobgp
## label: NULL
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## 0-9g/day    24     27.3         27.3
## 10-19       24     27.3         54.5
## 20-29       20     22.7         77.3
```

```
## 30+         20     22.7        100.0
## Total       88    100.0        100.0
## ====================================
```

5.2.2 Cross-tabulation

Exploring data is without cross-examing between two variables. When tabulating two categorical data, `xtab` comes in very handy. It can either simply cross-tab two variables or cross-tab between a numeric and a categorical data if both are treated as categorical data, or even cross-tab the whole dataset by a variable.

Let's try these with `esoph` dataset.

```r
# cross-tab agegp and alcgp
xtab(agegp, alcgp, esoph)
```

```
## ================================================================================
## Tabulation: agegp ~ alcgp
## --------------------------------------------------------------------------------
##       0-39g/day (r%) 40-79 (r%) 80-119 (r%) 120+ (r%) Total (r%) Chi.Square
## 25-34         4 26.7     4 26.7      3 20.0    4 26.7    15  100    1.00000
## 35-44         4 26.7     4 26.7      4 26.7    3 20.0    15  100
## 45-54         4 25.0     4 25.0      4 25.0    4 25.0    16  100
## 55-64         4 25.0     4 25.0      4 25.0    4 25.0    16  100
## 65-74         4 26.7     3 20.0      4 26.7    4 26.7    15  100
## 75+           3 27.3     4 36.4      2 18.2    2 18.2    11  100
## Total        23 26.1    23 26.1     21 23.9   21 23.9    88  100
##       F.Exact
## 25-34 1.00000
## 35-44
## 45-54
## 55-64
## 65-74
## 75+
## Total
## ================================================================================
```

```r
# cross-tab ncases and agegp
xtab(ncases, agegp, esoph)
```

```
## ==============================================================================================
## Tabulation: ncases ~ agegp
## ----------------------------------------------------------------------------------------------
##       25-34 (r%) 35-44 (r%) 45-54 (r%) 55-64  (r%) 65-74  (r%) 75+ (r%) Total
## 0        14 48.3    10 34.5     3 10.3     0   0.0     1   3.4   1  3.4    29
## 1         1  6.2     2 12.5     2 12.5     0   0.0     4  25.0   7 43.8    16
## 2         0  0.0     2 18.2     2 18.2     2  18.2     2  18.2   3 27.3    11
## 3         0  0.0     1 11.1     2 22.2     4  44.4     2  22.2   0  0.0     9
## 4         0  0.0     0  0.0     3 37.5     3  37.5     2  25.0   0  0.0     8
## 5         0  0.0     0  0.0     2 33.3     2  33.3     2  33.3   0  0.0     6
## 6         0  0.0     0  0.0     2 40.0     2  40.0     1  20.0   0  0.0     5
## 8         0  0.0     0  0.0     0  0.0     1 100.0     0   0.0   0  0.0     1
## 9         0  0.0     0  0.0     0  0.0     2 100.0     0   0.0   0  0.0     2
## 17        0  0.0     0  0.0     0  0.0     0   0.0     1 100.0   0  0.0     1
## Total    15 17.0    15 17.0    16 18.2    16  18.2    15  17.0  11 12.5    88
##       (r%) Chi.Square F.Exact
## 0      100    0.00001      NA
```

```
## 1      100
## 2      100
## 3      100
## 4      100
## 5      100
## 6      100
## 8      100
## 9      100
## 17     100
## Total  100
## ==========================================================================================
```

```
# cross-tab the whole dataset by agegp
xtab(esoph, agegp)
```

```
## ==========================================================================================
## Tabulation: agegp ~ agegp
## ------------------------------------------------------------------------------------------
##       25-34 (r%) 35-44 (r%) 45-54  (r%) 55-64  (r%) 65-74 (r%) 75+  (r%) Total
## 25-34    15  100     0    0     0   0.0     0   0.0     0    0   0   0.0    15
## 35-44     0    0    15  100     0   0.0     0   0.0     0    0   0   0.0    15
## 45-54     0    0     0    0    16 100.0     0   0.0     0    0   0   0.0    16
## 55-64     0    0     0    0     0   0.0    16 100.0     0    0   0   0.0    16
## 65-74     0    0     0    0     0   0.0     0   0.0    15  100   0   0.0    15
## 75+       0    0     0    0     0   0.0     0   0.0     0    0  11 100.0    11
## Total    15   17    15   17    16  18.2    16  18.2    15   17  11  12.5    88
##       (r%) Chi.Square F.Exact
## 25-34  100    0.00000 0.00000
## 35-44  100
## 45-54  100
## 55-64  100
## 65-74  100
## 75+    100
## Total  100
## ==========================================================================================
## ==========================================================================================
## Tabulation: alcgp ~ agegp
## ------------------------------------------------------------------------------------------
##           25-34 (r%) 35-44 (r%) 45-54 (r%) 55-64 (r%) 65-74 (r%) 75+ (r%) Total
## 0-39g/day     4 17.4     4 17.4     4 17.4     4 17.4     4 17.4   3 13.0    23
## 40-79         4 17.4     4 17.4     4 17.4     4 17.4     3 13.0   4 17.4    23
## 80-119        3 14.3     4 19.0     4 19.0     4 19.0     4 19.0   2  9.5    21
## 120+          4 19.0     3 14.3     4 19.0     4 19.0     4 19.0   2  9.5    21
## Total        15 17.0    15 17.0    16 18.2    16 18.2    15 17.0  11 12.5    88
##           (r%) Chi.Square F.Exact
## 0-39g/day  100    1.00000 1.00000
## 40-79      100
## 80-119     100
## 120+       100
## Total      100
## ==========================================================================================
## ==========================================================================================
## Tabulation: tobgp ~ agegp
## ------------------------------------------------------------------------------------------
##          25-34 (r%) 35-44 (r%) 45-54 (r%) 55-64 (r%) 65-74 (r%) 75+ (r%) Total
```

```
## 0-9g/day     4 16.7     4 16.7     4 16.7     4 16.7     4 16.7   4 16.7    24
## 10-19        4 16.7     4 16.7     4 16.7     4 16.7     4 16.7   4 16.7    24
## 20-29        3 15.0     4 20.0     4 20.0     4 20.0     4 20.0   1  5.0    20
## 30+          4 20.0     3 15.0     4 20.0     4 20.0     3 15.0   2 10.0    20
## Total       15 17.0    15 17.0    16 18.2    16 18.2    15 17.0  11 12.5    88
##          (r%) Chi.Square F.Exact
## 0-9g/day  100    0.99990 0.99991
## 10-19     100
## 20-29     100
## 30+       100
## Total     100
## ==========================================================================================
```

5.2.3 Number Summary

There are several number summary measures. However, `summ` function handpicks a set of commonly used measures and displays in a well-formatted manner. These measures include

1. Obs: Number of Observation
2. NA: Number of missing data (NA)
3. Mean: Arithmetic average
4. Std.Dev: Standard Deviation
5. Median: Median, 50% Quantile (Q50)
6. Q1: 25% Quantile (Q1)
7. Q3: 75% Quantile (Q3)
8. Min: Minimum
9. Max: Maximum
10. CV: Coefficient of Variation, (SD / Mean) * 100
11. Normal.test: Shapiro-Wilk Normality Test

This function also lets you summarize one variable, several variables or the whole dataset.

Let's summarize `infert` dataset.

```r
# summary measure of age
summ(age, infert)
```

```
## ================================================================
## Number Summary: age
## label: NULL
## ----------------------------------------------------------------
##     Obs. NA. Mean Std.Dev Median Q1   Q3 Min Max  CV. Normal.test
## age  248   0 31.5     5.3     31 28 35.2  21  44 16.7     0.00034
## ================================================================
```

```r
# summary measure of age, parity, induced, case
summ(c(age, parity, induced, case), infert)
```

```
## ====================================================================
## Number Summary: age | parity | induced | case
## --------------------------------------------------------------------
##         Obs. NA. Mean Std.Dev Median Q1   Q3 Min Max   CV. Normal.test
## age      248   0 31.5     5.3     31 28 35.2  21  44  16.7     0.00034
## parity   248   0  2.1     1.3      2  1  3.0   1   6  59.8     0.00000
## induced  248   0  0.6     0.7      0  0  1.0   0   2 129.0     0.00000
## case     248   0  0.3     0.5      0  0  1.0   0   1 141.3     0.00000
## ====================================================================
```

```r
# summary measure of infert
summ(infert)
```

```
## ==============================================================================
## Number Summary: age | parity | induced | case | spontaneous | pooled.stratum
## ------------------------------------------------------------------------------
##                Obs. NA. Mean Std.Dev Median Q1   Q3 Min Max   CV. Normal.test
## age             248   0 31.5     5.3     31 28 35.2  21  44  16.7     0.00034
## parity          248   0  2.1     1.3      2  1  3.0   1   6  59.8     0.00000
## induced         248   0  0.6     0.7      0  0  1.0   0   2 129.0     0.00000
## case            248   0  0.3     0.5      0  0  1.0   0   1 141.3     0.00000
## spontaneous     248   0  0.6     0.7      0  0  1.0   0   2 127.0     0.00000
## pooled.stratum  248   0 33.6    17.3     36 19 48.2   1  63  51.4     0.00000
## ==============================================================================
```

5.2.4 Grouped Summary Measures

Summary measures can be disaggregated by a categorical variable. With `summBy` function, p values from tests such as student's t-test and ANOVA test based on number of groups will be automatically displayed along with some important summary measures. These summary measures are relatively fewer, compared to the outputs from `summ`.

Similar to previous functions, `summBy` can also be used with two variables as well as summarization of the whole dataset by a categorical data.

Let's demonstrate with `infert` dataset.

```r
# summary measures of age by education
summBy(age, education, infert)
```

```
## ==========================================================================
## Number Summary: age
## by categories of: education
## --------------------------------------------------------------------------
##         Obs. NA. Mean Std.Dev Median   Q1   Q3 Normal.test   ANOVA K-Wallis
## 0-5yrs    12   0 35.2     6.3   36.5 32.0 39.8     0.02842 0.00000  0.00000
## 6-11yrs  120   0 32.9     5.4   33.0 28.8 36.2     0.11209
## 12+ yrs  116   0 29.7     4.4   29.0 26.0 32.5     0.00076
## ==========================================================================
```

```r
# summary measures of several variables by education
summBy(c(age, parity, induced), education, infert)
```

```
## ==================================================================================
## Number Summary: age | parity | induced
## by categories of: education
## ----------------------------------------------------------------------------------
##                 Obs. NA. Mean Std.Dev Median   Q1   Q3 Normal.test   ANOVA
## age.0-5yrs        12   0 35.2     6.3   36.5   32 39.8     0.02842 0.00000
## age.6-11yrs      120   0 32.9     5.4     33 28.8 36.2     0.11209
## age.12+ yrs      116   0 29.7     4.4     29   26 32.5     0.00076
## age.
## parity.0-5yrs     12   0  4.2     2.1      5  3.2    6     0.00263 0.00000
## parity.6-11yrs   120   0  2.1     1.1      2    1    3     0.00000
## parity.12+ yrs   116   0  1.9     1.1      2    1    2     0.00000
## parity.
## induced.0-5yrs    12   0  1.2     0.9    1.5    0    2     0.00197 0.00573
```

```
## induced.6-11yrs  120   0  0.5     0.7      0    0    1     0.00000
## induced.12+ yrs  116   0  0.6     0.7      0    0    1     0.00000
## induced.
##                 K-Wallis
## age.0-5yrs       0.00000
## age.6-11yrs
## age.12+ yrs
## age.
## parity.0-5yrs    0.00033
## parity.6-11yrs
## parity.12+ yrs
## parity.
## induced.0-5yrs   0.01201
## induced.6-11yrs
## induced.12+ yrs
## induced.
## ================================================================================
```

```
# wummary measures of the whole dataset by case
summBy(infert, case)
```

```
## ================================================================================
## Number Summary: age | parity | induced | case | spontaneous | pooled.stratum
## by categories of: case
## --------------------------------------------------------------------------------
##                  Obs. NA. Mean Std.Dev Median Q1   Q3 Normal.test  t-test
## age.0             165   0 31.5     5.3     31 28   35     0.00635 0.95598
## age.1              83   0 31.5     5.3     31 28 35.5     0.15788
## age.
## parity.0          165   0  2.1     1.2      2  1    3     0.00000 0.89009
## parity.1           83   0  2.1     1.3      2  1    3     0.00000
## parity.
## induced.0         165   0  0.6     0.7      0  0    1     0.00000 0.79006
## induced.1          83   0  0.6     0.7      0  0    1     0.00000
## induced.
## case.0            165   0    0       0      0  0    0          NA      NA
## case.1             83   0    1       0      1  1    1          NA
## case.
## spontaneous.0     165   0  0.4     0.6      0  0    1     0.00000 0.00000
## spontaneous.1      83   0    1     0.8      1  0    2     0.00000
## spontaneous.
## pooled.stratum.0  165   0 33.5    17.3     36 19   48     0.00003 0.93949
## pooled.stratum.1   83   0 33.7    17.4     36 19 48.5     0.00567
## pooled.stratum.
##                  Wilcoxon
## age.0             0.95427
## age.1
## age.
## parity.0          0.93768
## parity.1
## parity.
## induced.0         0.79527
## induced.1
## induced.
## case.0            0.00000
```

```
## case.1
## case.
## spontaneous.0     0.00000
## spontaneous.1
## spontaneous.
## pooled.stratum.0  0.93943
## pooled.stratum.1
## pooled.stratum.
## ================================================================================
```

5.3 Formatting model outputs

As writing of this session, the functions `fModelOutput` and `testModelFit` can be applied to outputs from logistic regression, both unadjusted and adjusted analyses.

Basically, the outputs from the logit model is formated in `fModelOutput` jnto commonly reported statistics. That includes

1. Point Estimate: un-adjusted or Adjusted Odds Ratio
2. Interval Estimate: 95% confidence intervals
3. wald's P-value
4. Log-likelihood value
5. AIC

If multivariable model is fed, then both un-adjusted and adjusted statistics are automatically calcaulated and displayed in a well-formatted table.

Let's try unadjusted analysis using `infert` dataset.

```r
## data infert
str(infert)
```

```
## 'data.frame':    248 obs. of  8 variables:
##  $ education     : Factor w/ 3 levels "0-5yrs","6-11yrs",..: 1 1 1 1 2 2 2 2 2 2 ...
##  $ age           : num  26 42 39 34 35 36 23 32 21 28 ...
##  $ parity        : num  6 1 6 4 3 4 1 2 1 2 ...
##  $ induced       : num  1 1 2 2 1 2 0 0 0 0 ...
##  $ case          : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ spontaneous   : num  2 0 0 0 1 1 0 0 1 0 ...
##  $ stratum       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ pooled.stratum: num  3 1 4 2 32 36 6 22 5 19 ...
```

```r
## fit the model: case ~ education
logit.edu <- glm(case ~ education, data = infert, family = binomial)
## display summary
summary(logit.edu)
```

```
##
## Call:
## glm(formula = case ~ education, family = binomial, data = infert)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -0.9053  -0.9053  -0.9005   1.4765   1.4823
##
## Coefficients:
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -6.931e-01  6.124e-01  -1.132    0.258
## education6-11yrs  4.477e-15  6.423e-01   0.000    1.000
## education12+ yrs  1.290e-02  6.431e-01   0.020    0.984
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 316.17  on 247  degrees of freedom
## Residual deviance: 316.17  on 245  degrees of freedom
## AIC: 322.17
##
## Number of Fisher Scoring iterations: 4
```

```r
## format model output
fModelOutput(logit.edu)
```

```
## ========================================================
## Output of Logistic Regression
## Number of obs: 248
## --------------------------------------------------------
##                  + ------ uOR (95% CI) Pr(>|z|) ------ +
## education6-11yrs |     1.0 (0.3 - 3.9)  1.00000        |
## education12+ yrs |     1.0 (0.3 - 4.0)  0.98399        |
## ========================================================
## (Log-likelihood: -158.1)
## (AIC: 322.2)
```

```r
## test over all fittness
testModelFit(logit.edu)
```

```
##
##      Measure of model fit
##
## X-square = 0.002289413, df = 2, p-value = 0.9988559
##
## Interpretation:
##  The model as a whole does not fit significantly better than an empty model.
```

Let's try adjusted analysis using `infert` dataset.

```r
## fit the model: case ~ education
logit.multi <- glm(case ~ education + age + factor(induced), data = infert,
                 family = binomial)
## display summary
summary(logit.multi)
```

```
##
## Call:
## glm(formula = case ~ education + age + factor(induced), family = binomial,
##     data = infert)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -0.9423  -0.9049  -0.8915   1.4717   1.5087
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -0.859213   1.186828  -0.724    0.469
## education6-11yrs  0.046034   0.663861   0.069    0.945
## education12+ yrs  0.061719   0.678611   0.091    0.928
## age               0.002922   0.027433   0.107    0.915
## factor(induced)1  0.044295   0.315476   0.140    0.888
## factor(induced)2  0.110580   0.399789   0.277    0.782
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 316.17  on 247  degrees of freedom
## Residual deviance: 316.08  on 242  degrees of freedom
## AIC: 328.08
```

```
##
## Number of Fisher Scoring iterations: 4
```

```r
## format model output
fModelOutput(logit.multi)
```

```
## ====================================================
## Output of unadjusted Logistic Regression
## Number of obs: 248
## ----------------------------------------------------
##                  + --- uOR (95% CI)  Pr(>|z|) --- +
## education6-11yrs |  1.0 (0.3 - 3.9)   1.00000   . |
## education12+ yrs |  1.0 (0.3 - 4.0)   0.98399   . |
## age              |  1.0 (1.0 - 1.1)   0.95566   . |
## factor(induced)1 |  1.0 (0.6 - 1.9)   0.89033   . |
## factor(induced)2 |  1.1 (0.5 - 2.3)   0.79426   . |
## ====================================================
## ====================================================
## Output of Adjusted Logistic Regression
## Number of obs: 248
## ----------------------------------------------------
##                  + --- aOR (95% CI)  Pr(>|z|) --- +
## education6-11yrs |  1.0 (0.3 - 4.3)   0.94472   . |
## education12+ yrs |  1.1 (0.3 - 4.4)   0.92753   . |
## age              |  1.0 (1.0 - 1.1)   0.91517   . |
## factor(induced)1 |  1.0 (0.6 - 1.9)   0.88834   . |
## factor(induced)2 |  1.1 (0.5 - 2.4)   0.78209   . |
## ====================================================
## (Log-likelihood: -158.0)
## (AIC: 328.1)
```

```r
## test over all fittness
testModelFit(logit.multi)
```

```
##
##      Measure of model fit
##
## X-square = 0.08893515, df = 5, p-value = 0.9998784
##
## Interpretation:
##  The model as a whole does not fit significantly better than an empty model.
```

Let's try one more example from IRDE website.

> LOGIT REGRESSION | R DATA ANALYSIS EXAMPLES. UCLA: Statistical Consulting Group. from https://stats.idre.ucla.edu/r/dae/logit-regression/ (accessed February 13, 2020).

```r
## reading data
mydata <- read.csv("https://stats.idre.ucla.edu/stat/data/binary.csv")

mydata$rank <- factor(mydata$rank)
mylogit <- glm(admit ~ gre + gpa + rank, data = mydata, family = "binomial")

## Format model output
fModelOutput(mylogit)
```

```
## ========================================
```

```
## Output of unadjusted Logistic Regression
##
## Number of obs: 400
## ----------------------------------------
##       + --- uOR (95% CI)  Pr(>|z|) --- +
## gre   |  1.0 (1.0 - 1.0)   0.00028 *** |
## gpa   |  2.9 (1.6 - 5.2)   0.00044 *** |
## rank2 |  0.5 (0.3 - 0.9)   0.01488   * |
## rank3 |  0.3 (0.1 - 0.5)   0.00005 *** |
## rank4 |  0.2 (0.1 - 0.4)   0.00004 *** |
## ========================================
## ========================================
## Output of Adjusted Logistic Regression
## Number of obs: 400
## ----------------------------------------
##       + --- aOR (95% CI)  Pr(>|z|) --- +
## gre   |  1.0 (1.0 - 1.0)   0.03847   * |
## gpa   |  2.2 (1.2 - 4.3)   0.01539   * |
## rank2 |  0.5 (0.3 - 0.9)   0.03283   * |
## rank3 |  0.3 (0.1 - 0.5)   0.00010 *** |
## rank4 |  0.2 (0.1 - 0.5)   0.00020 *** |
## ========================================
## (Log-likelihood: -229.3)
## (AIC: 470.5)
```

```r
testModelFit(mylogit)
```

```
##
##      Measure of model fit
##
## X-square = 41.45903, df = 5, p-value = 7.578194e-08
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

6 Workflow in `mStats` package

In this section, the workflow of mStats package is demonstrated using a real world dataset published in a internationally peer-reviewed journal. The dataset has been publically available for download from this link.

> Martelli G, Antonucci R, Mukurasi A, Zepherine H, Nöstlinger C (2019) Adherence to antiretroviral treatment among children and adolescents in Tanzania: Comparison between pill count and viral load outcomes in a rural context of Mwanza region. PLoS ONE 14(3): e0214014. https://doi.org/10.1371/journal.pone.0214014

6.1 piping `%>%` from `magrittr`

The `mStats` has been written in a compatible way of working with piping function `%>%` from `magrittr` package. When the argument `data` or equivalent is in the first position, `%>%` can be used as usual, by omitting the first `data` arugment.

If such argument is not in the first postion, the dot `.` can be put to join the statements.

Let’s demonstrate using HIV dataset from the paper described above.

6.2 Milestone Project: Objectives

The aim of the milestone project is to appreciate and understand the efficient workflow of the `mStats` package in combination with piping `%>%` from `magrittr`.

Objectives

1. To process dataset with the table 1 and 2 format in mind.
2. To create Table 1 and 2 using what we have learnt so far.

6.3 Milestone Project: Code Demonstration

we will re-create Table 1 from the dataset.

As you can see in the codebook, almost 40% of VLRcat and AdherenceR have missing values.

Meta-data for the dataset can be found here.

```
## loading the mStats and magrittr
library(mStats)
library(magrittr)
## clear the workspace
clear()
```

```r
## logging the console outputs
ilog("./data/mStats_Demo.txt", append = FALSE)
```

```
##       log: C:/MEGA/GitHub/R/Dev_Folder/mStats_Guide/./data/mStats_Demo.txt
##   open on: 2020-02-15 12:14:29
##      note: replaced
## ----------------------------------------------------------------------
```

```r
## reading xls format using readxl
hiv <- readxl::read_excel("./data/pone.0214014.s001.xls")
## checking data structure
str(hiv)
```

```
## Classes 'tbl_df', 'tbl' and 'data.frame':    72 obs. of  24 variables:
##  $ ID         : num  1 2 3 4 5 6 7 8 9 10 ...
##  $ TB         : chr  "n" "n" "n" "n" ...
##  $ Sex        : chr  "F" "F" "M" "F" ...
##  $ DiagnHIV   : POSIXct, format: "2013-12-18" "2012-09-05" ...
##  $ District   : num  2 1 1 1 1 1 1 2 1 1 ...
##  $ WHOstage   : num  3 1 3 3 3 3 2 2 2 2 ...
##  $ ARVregimen : num  2 1 5 1 4 1 1 5 1 3 ...
##  $ Nregimens  : num  1 1 3 1 1 1 1 3 1 1 ...
##  $ ARVStart   : POSIXct, format: "2014-01-07" "2012-08-21" ...
##  $ ARVtablet  : num  3 2 4 4 9 2 1.5 10 5 6 ...
##  $ CTX        : chr  "n" "n" "n" "y" ...
##  $ Vitamin    : chr  "n" "n" "y" "y" ...
##  $ IPT        : chr  "n" "n" "n" "y" ...
##  $ Nutrition  : num  1 1 3 1 1 1 2 1 2 2 ...
##  $ Infection  : num  1 2 3 1 1 1 0 7 1 1 ...
##  $ CD4        : num  530 1255 13 1565 1390 ...
##  $ VLcat      : num  0 0 2 0 2 1 1 2 2 0 ...
##  $ Adherence  : num  1 1 1 1 2 1 1 2 1 1 ...
##  $ RelativeHIV: chr  "y" "y" "n" "n" ...
##  $ Relation   : num  1 1 3 4 1 1 1 1 3 1 ...
##  $ VLRcat     : num  0 3 NA NA 3 0 0 1 NA 0 ...
##  $ AdherenceR : num  1 1 NA NA 1 1 1 1 NA 1 ...
##  $ Literacy   : chr  "n" "y" "y" "n" ...
##  $ SiblingsHIV: chr  "n" "n" "n" "n" ...
```

```r
## codebook
codebook(hiv)
```

```
## ========================================================
## Codebook: hiv
##    label: NULL
## --------------------------------------------------------
##      VARS_NAME LABEL DATA_TYPE OBS_COUNT NA_COUNT NA_(%)
## 1           ID  NULL   numeric        72        0    0 %
## 2           TB  NULL character        72        0    0 %
## 3          Sex  NULL character        72        0    0 %
## 4     DiagnHIV  NULL   numeric        72        0    0 %
## 5     District  NULL   numeric        72        0    0 %
## 6     WHOstage  NULL   numeric        72        0    0 %
## 7   ARVregimen  NULL   numeric        72        0    0 %
## 8    Nregimens  NULL   numeric        72        0    0 %
## 9     ARVStart  NULL   numeric        72        0    0 %
```

```
## 10   ARVtablet  NULL   numeric        72        0    0 %
## 11         CTX  NULL character        72        0    0 %
## 12     Vitamin  NULL character        72        0    0 %
## 13         IPT  NULL character        72        0    0 %
## 14   Nutrition  NULL   numeric        72        0    0 %
## 15   Infection  NULL   numeric        72        0    0 %
## 16         CD4  NULL   numeric        72        0    0 %
## 17       VLcat  NULL   numeric        72        0    0 %
## 18   Adherence  NULL   numeric        72        0    0 %
## 19 RelativeHIV  NULL character        72        0    0 %
## 20    Relation  NULL   numeric        72        0    0 %
## 21      VLRcat  NULL   numeric        45       27 37.5 %
## 22  AdherenceR  NULL   numeric        45       27 37.5 %
## 23    Literacy  NULL character        72        0    0 %
## 24 SiblingsHIV  NULL character        72        0    0 %
## ========================================================
```

```
## Table 1: variables needed
## Sex, TB, District, WHOstage, ARVregimen, ARVStart, ARVtablet
## Nutrition, CD4, Relation, RelativeHIV, SiblingsHIV, Literacy
hiv.tables <- hiv %>%

    # remove the rest of variables
    lose(DiagnHIV, CTX, Vitamin, IPT, Infection, Adherence, AdherenceR) %>%

    # recoding values
    recode(Sex, c("F", "M"), c("Female", "Male")) %>%
    recode(TB, c("y", "n"), c("Positive", "Negative")) %>%
    recode(District, c(1, 2), c("Misungwi", "others")) %>%
    recode(WHOstage, c(1,2,3,4), c("I", "II", "III", "IV")) %>%

    # generate ARV for abbreviations
    generate(ARV, ARVregimen) %>%
    # recoding ARV
    recode(ARV,
           c(1, 2, 3,
             4, 5, 6, 7),
           c("3TC+AZT+NVP", "3TC+AZT+EFV", "ABC+3CT+EFV",
             "ABC+3TC+LPV/r", "ABC+3TC+LPV/r", "TDF+FDC+ATV/r",
             "ABC+3TC+ATV/r")) %>%

    # new variable for classifying PI and Non-PI based regimens
    generate(ARVpi, ifelse(ARVregimen < 4, "Non-PI", "PI")) %>%

    # new variable to calculate time on ART: date difference / 365.25 = years
    generate(timeART, as.numeric(as.Date("2017-12-31") - as.Date(ARVStart)) / 365.25) %>%
    # egen timeART.grp
    egen(timeART, c(2.0, 6.0, 9.0), c("3-24M", "25M-5Yr", "6-8Yr", "9+Yr"),
         timeART.grp) %>%

    # number of ARV tablets
egen(ARVtablet, c(2.5, 4.5, 6),
     c("<=2.4", "2.5-4.4", "4.5-5.9", "6+"), ARVtab.grp) %>%
```

```
    # recode nutrition: 1 = normal, 2 = moderate, 3 = severe malnourished
    recode(Nutrition, c(1,2,3), c("Normal", "Moderate", "Severe")) %>%

    # egen CD4 group: cut-off point 351
    egen(CD4, c(0, 351), c("<=350", "351+"), cd4.grp) %>%
    # egen CD4 group: cut-point 500
    egen(CD4, c(0, 501), c("<=500", "501+"), cd4.grp2) %>%

    # recode relation: 1 = Mother, 2 = Father, 3 = Relative, 4 = Other
    recode(Relation, c(1,2,3,4), c("Mother", "Father", "Relative", "Others")) %>%
    # recode HIV Caregiver
    recode(RelativeHIV, c("y", "n"), c("Yes", "No")) %>%
    # recode siblings HIV
    recode(SiblingsHIV, c("y", "n"), c("Yes", "No")) %>%
    # recode caregiver literacy
    recode(Literacy, c("y", "n"), c("Yes", "No")) %>%

    # new variable for viral suppression: vlSuppress
    generate(vlSuppress, ifelse(VLcat < 2, 1, 0)) %>%

    # keep all we want for Table 1 and 2
    keep(Sex, TB, District, WHOstage, ARVpi, ARV, timeART.grp, ARVtab.grp,
         Nutrition, cd4.grp, Relation, RelativeHIV, SiblingsHIV, Literacy,
         timeART, Nregimens, cd4.grp2, vlSuppress) %>%

    # variable labels
    labelVars(c(Sex, TB,
                District, WHOstage,
                ARVpi, ARV, timeART.grp,
                ARVtab.grp, Nutrition,
                cd4.grp, Relation, RelativeHIV,
                SiblingsHIV, Literacy,
                timeART, Nregimens, cd4.grp2, vlSuppress),
              c("Gender", "History of TB",
                "District: 1=Misungwi,2=others", "WHO Clinical Staging",
                "ART PI-NonPI", "ARV regimens", "Time on ART Category",
                "number of ARV tablet/day", "Nutritional Status",
                "CD4 group: 350", "Relation with Caregiver", "HIV+ Caregiver",
                "HIV+ Siblings", "Literacy of caregiver",
                "Time on ART Years", "Previous Regimens",
                "CD4 group: 500", "Viral Suppression"))
```

```
## (7 variables deleted: DiagnHIV | CTX | Vitamin | IPT | Infection | Adherence | A
## dherenceR)
## (39 values recoded | 'F' => 'Female')
## (33 values recoded | 'M' => 'Male')
## (8 values recoded | 'y' => 'Positive')
## (64 values recoded | 'n' => 'Negative')
## (56 values recoded | '1' => 'Misungwi')
## (16 values recoded | '2' => 'others')
## (6 values recoded | '1' => 'I')
## (22 values recoded | '2' => 'II')
## (38 values recoded | '3' => 'III')
## (6 values recoded | '4' => 'IV')
```

```
## (72 values generated)
## (Expression used: 'with(., ARVregimen)')
## (34 values recoded | '1' => '3TC+AZT+NVP')
## (6 values recoded | '2' => '3TC+AZT+EFV')
## (15 values recoded | '3' => 'ABC+3CT+EFV')
## (6 values recoded | '4' => 'ABC+3TC+LPV/r')
## (7 values recoded | '5' => 'ABC+3TC+LPV/r')
## (1 values recoded | '6' => 'TDF+FDC+ATV/r')
## (3 values recoded | '7' => 'ABC+3TC+ATV/r')
## (72 values generated)
## (Expression used: 'with(., ifelse(ARVregimen < 4, "Non-PI", "PI"))')
## (72 values generated)
## (Expression used: 'with(., as.numeric(as.Date("2017-12-31") - as.Date(ARVStart))
## /365.25)')
## (72 values generated with labels: 3-24M, 25M-5Yr, 6-8Yr, 9+Yr)
## ('timeART.grp' created & appended to dataframe '.')
## (72 values generated with labels: <=2.4, 2.5-4.4, 4.5-5.9, 6+)
## ('ARVtab.grp' created & appended to dataframe '.')
## (45 values recoded | '1' => 'Normal')
## (19 values recoded | '2' => 'Moderate')
## (8 values recoded | '3' => 'Severe')
## (72 values generated with labels: <=350, 351+)
## ('cd4.grp' created & appended to dataframe '.')
## (72 values generated with labels: <=500, 501+)
## ('cd4.grp2' created & appended to dataframe '.')
## (37 values recoded | '1' => 'Mother')
## (9 values recoded | '2' => 'Father')
## (21 values recoded | '3' => 'Relative')
## (5 values recoded | '4' => 'Others')
## (45 values recoded | 'y' => 'Yes')
## (27 values recoded | 'n' => 'No')
## (13 values recoded | 'y' => 'Yes')
## (59 values recoded | 'n' => 'No')
## (43 values recoded | 'y' => 'Yes')
## (29 values recoded | 'n' => 'No')
## (72 values generated)
## (Expression used: 'with(., ifelse(VLcat < 2, 1, 0))')
## (18 variables kept: TB | Sex | District | WHOstage | Nregimens | Nutrition | Rel
## ativeHIV | Relation | Literacy | SiblingsHIV | ARV | ARVpi | timeART | timeART.g
## rp | ARVtab.grp | cd4.grp | cd4.grp2 | vlSuppress)
## (Variable: labelled 'Sex' as 'Gender')
## (Variable: labelled 'TB' as 'History of TB')
## (Variable: labelled 'District' as 'District: 1=Misungwi,2=others')
## (Variable: labelled 'WHOstage' as 'WHO Clinical Staging')
## (Variable: labelled 'ARVpi' as 'ART PI-NonPI')
## (Variable: labelled 'ARV' as 'ARV regimens')
## (Variable: labelled 'timeART.grp' as 'Time on ART Category')
## (Variable: labelled 'ARVtab.grp' as 'number of ARV tablet/day')
## (Variable: labelled 'Nutrition' as 'Nutritional Status')
## (Variable: labelled 'cd4.grp' as 'CD4 group: 350')
## (Variable: labelled 'Relation' as 'Relation with Caregiver')
## (Variable: labelled 'RelativeHIV' as 'HIV+ Caregiver')
## (Variable: labelled 'SiblingsHIV' as 'HIV+ Siblings')
## (Variable: labelled 'Literacy' as 'Literacy of caregiver')
```

```
## (Variable: labelled 'timeART' as 'Time on ART Years')
## (Variable: labelled 'Nregimens' as 'Previous Regimens')
## (Variable: labelled 'cd4.grp2' as 'CD4 group: 500')
## (Variable: labelled 'vlSuppress' as 'Viral Suppression')
```

```r
# check codebook
codebook(hiv.tables)
```

```
## ================================================================================
## Codebook: hiv.tables
##    label: NULL
## --------------------------------------------------------------------------------
##      VARS_NAME                          LABEL DATA_TYPE OBS_COUNT NA_COUNT
## 1           TB                  History of TB character        72        0
## 2          Sex                         Gender character        72        0
## 3     District District: 1=Misungwi,2=others character        72        0
## 4     WHOstage          WHO Clinical Staging character        72        0
## 5    Nregimens             Previous Regimens   numeric        72        0
## 6    Nutrition            Nutritional Status character        72        0
## 7  RelativeHIV                HIV+ Caregiver character        72        0
## 8     Relation       Relation with Caregiver character        72        0
## 9     Literacy         Literacy of caregiver character        72        0
## 10 SiblingsHIV                 HIV+ Siblings character        72        0
## 11         ARV                  ARV regimens character        72        0
## 12       ARVpi                  ART PI-NonPI    factor        72        0
## 13     timeART             Time on ART Years   numeric        72        0
## 14 timeART.grp          Time on ART Category    factor        72        0
## 15  ARVtab.grp      number of ARV tablet/day    factor        72        0
## 16     cd4.grp                CD4 group: 350    factor        72        0
## 17    cd4.grp2                CD4 group: 500    factor        72        0
## 18  vlSuppress             Viral Suppression   numeric        72        0
##    NA_(%)
## 1     0 %
## 2     0 %
## 3     0 %
## 4     0 %
## 5     0 %
## 6     0 %
## 7     0 %
## 8     0 %
## 9     0 %
## 10    0 %
## 11    0 %
## 12    0 %
## 13    0 %
## 14    0 %
## 15    0 %
## 16    0 %
## 17    0 %
## 18    0 %
## ================================================================================
```

```r
## Table 1: tabulation
tab(hiv.tables)
```

```
## ====================================
```

```
## Tabulation: TB
## label: History of TB
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## Negative    64     88.9         88.9
## Positive     8     11.1        100.0
## Total       72    100.0        100.0
## ====================================
## ==================================
## Tabulation: Sex
## label: Gender
## ----------------------------------
##        Freq. Percent. Cum.Percent.
## Female    39     54.2         54.2
## Male      33     45.8        100.0
## Total     72    100.0        100.0
## ==================================
## ====================================
## Tabulation: District
## label: District: 1=Misungwi,2=others
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## Misungwi    56     77.8         77.8
## others      16     22.2        100.0
## Total       72    100.0        100.0
## ====================================
## ================================
## Tabulation: WHOstage
## label: WHO Clinical Staging
## --------------------------------
##       Freq. Percent. Cum.Percent.
## I         6      8.3          8.3
## II       22     30.6         38.9
## III      38     52.8         91.7
## IV        6      8.3        100.0
## Total    72    100.0        100.0
## ================================
## ====================================
## Tabulation: Nutrition
## label: Nutritional Status
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## Moderate    19     26.4         26.4
## Normal      45     62.5         88.9
## Severe       8     11.1        100.0
## Total       72    100.0        100.0
## ====================================
## ================================
## Tabulation: RelativeHIV
## label: HIV+ Caregiver
## --------------------------------
##       Freq. Percent. Cum.Percent.
## No       27     37.5         37.5
## Yes      45     62.5        100.0
```

```
## Total    72    100.0        100.0
## ================================
## ====================================
## Tabulation: Relation
## label: Relation with Caregiver
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## Father       9     12.5         12.5
## Mother      37     51.4         63.9
## Others       5      6.9         70.8
## Relative    21     29.2        100.0
## Total       72    100.0        100.0
## ====================================
## ================================
## Tabulation: Literacy
## label: Literacy of caregiver
## --------------------------------
##       Freq. Percent. Cum.Percent.
## No       29     40.3         40.3
## Yes      43     59.7        100.0
## Total    72    100.0        100.0
## ================================
## ================================
## Tabulation: SiblingsHIV
## label: HIV+ Siblings
## --------------------------------
##       Freq. Percent. Cum.Percent.
## No       59     81.9         81.9
## Yes      13     18.1        100.0
## Total    72    100.0        100.0
## ================================
## ========================================
## Tabulation: ARV
## label: ARV regimens
## ----------------------------------------
##               Freq. Percent. Cum.Percent.
## 3TC+AZT+EFV       6      8.3          8.3
## 3TC+AZT+NVP      34     47.2         55.6
## ABC+3CT+EFV      15     20.8         76.4
## ABC+3TC+ATV/r     3      4.2         80.6
## ABC+3TC+LPV/r    13     18.1         98.6
## TDF+FDC+ATV/r     1      1.4        100.0
## Total            72    100.0        100.0
## ========================================
## ==================================
## Tabulation: ARVpi
## label: ART PI-NonPI
## ----------------------------------
##        Freq. Percent. Cum.Percent.
## Non-PI    55     76.4         76.4
## PI        17     23.6        100.0
## Total     72    100.0        100.0
## ==================================
## ====================================
```

```
## Tabulation: timeART.grp
## label: Time on ART Category
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## 3-24M      12     16.7         16.7
## 25M-5Yr    46     63.9         80.6
## 6-8Yr      10     13.9         94.4
## 9+Yr        4      5.6        100.0
## Total      72    100.0        100.0
## ====================================
## ====================================
## Tabulation: ARVtab.grp
## label: number of ARV tablet/day
## ------------------------------------
##          Freq. Percent. Cum.Percent.
## <=2.4      30     41.7         41.7
## 2.5-4.4    10     13.9         55.6
## 4.5-5.9    13     18.1         73.6
## 6+         19     26.4        100.0
## Total      72    100.0        100.0
## ====================================
## ==================================
## Tabulation: cd4.grp
## label: CD4 group: 350
## ---------------------------------
##        Freq. Percent. Cum.Percent.
## <=350     7      9.7          9.7
## 351+     65     90.3        100.0
## Total    72    100.0        100.0
## =================================
## =================================
## Tabulation: cd4.grp2
## label: CD4 group: 500
## ---------------------------------
##        Freq. Percent. Cum.Percent.
## <=500    12     16.7         16.7
## 501+     60     83.3        100.0
## Total    72    100.0        100.0
## =================================
```

```r
## Summary measures (not included in Table 1)
summ(hiv.tables)
```

```
## ==========================================================================
## Number Summary: Nregimens | timeART | vlSuppress
## --------------------------------------------------------------------------
##            Obs. NA. Mean Std.Dev Median  Q1  Q3 Min  Max  CV. Normal.test
## Nregimens    72   0  1.5     0.8      1 1.0 2.0 1.0  4.0 52.7     0.00000
## timeART      72   0  4.3     2.5      4 2.5 5.6 0.6 11.6 57.7     0.00130
## vlSuppress   72   0  0.6     0.5      1 0.0 1.0 0.0  1.0 78.0     0.00000
## ==========================================================================
## (labels: )
## (Nregimens: Previous Regimens)
## (timeART: Time on ART Years)
## (vlSuppress: Viral Suppression)
```

```r
## Table 2: cross-tabulation by Viral Suppression
xtab(hiv.tables, vlSuppress)
```

```
## ======================================================
## Tabulation: TB ~ vlSuppress
## ------------------------------------------------------
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Negative 23 35.9 41 64.1    64  100    0.43858 0.46257
## Positive  4 50.0  4 50.0     8  100
## Total    27 37.5 45 62.5    72  100
## ======================================================
## (Labels:)
## (TB: History of TB)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: Sex ~ vlSuppress
## ----------------------------------------------------
##         0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Female 14 35.9 25 64.1    39  100    0.76010 0.81025
## Male   13 39.4 20 60.6    33  100
## Total  27 37.5 45 62.5    72  100
## ====================================================
## (Labels:)
## (Sex: Gender)
## (vlSuppress: Viral Suppression)
## ======================================================
## Tabulation: District ~ vlSuppress
## ------------------------------------------------------
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Misungwi 25 44.6 31 55.4    56  100    0.01917 0.02122
## others    2 12.5 14 87.5    16  100
## Total    27 37.5 45 62.5    72  100
## ======================================================
## (Labels:)
## (District: District: 1=Misungwi,2=others)
## (vlSuppress: Viral Suppression)
## ===================================================
## Tabulation: WHOstage ~ vlSuppress
## ---------------------------------------------------
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## I      3 50.0  3 50.0     6  100    0.86637 0.89569
## II     9 40.9 13 59.1    22  100
## III   13 34.2 25 65.8    38  100
## IV     2 33.3  4 66.7     6  100
## Total 27 37.5 45 62.5    72  100
## ===================================================
## (Labels:)
## (WHOstage: WHO Clinical Staging)
## (vlSuppress: Viral Suppression)
## ======================================================
## Tabulation: Nutrition ~ vlSuppress
## ------------------------------------------------------
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Moderate  6 31.6 13 68.4    19  100    0.81701 0.87557
```

```
## Normal   18 40.0 27 60.0    45  100
## Severe    3 37.5  5 62.5     8  100
## Total    27 37.5 45 62.5    72  100
## ======================================================
## (Labels:)
## (Nutrition: Nutritional Status)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: RelativeHIV ~ vlSuppress
## ----------------------------------------------------
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## No    12 44.4 15 55.6    27  100    0.34578 0.45165
## Yes   15 33.3 30 66.7    45  100
## Total 27 37.5 45 62.5    72  100
## ====================================================
## (Labels:)
## (RelativeHIV: HIV+ Caregiver)
## (vlSuppress: Viral Suppression)
## ======================================================
## Tabulation: Relation ~ vlSuppress
## ------------------------------------------------------
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Father    4 44.4  5 55.6     9  100    0.55818 0.50778
## Mother   11 29.7 26 70.3    37  100
## Others    2 40.0  3 60.0     5  100
## Relative 10 47.6 11 52.4    21  100
## Total    27 37.5 45 62.5    72  100
## ======================================================
## (Labels:)
## (Relation: Relation with Caregiver)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: Literacy ~ vlSuppress
## ----------------------------------------------------
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## No    11 37.9 18 62.1    29  100    0.95053 1.00000
## Yes   16 37.2 27 62.8    43  100
## Total 27 37.5 45 62.5    72  100
## ====================================================
## (Labels:)
## (Literacy: Literacy of caregiver)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: SiblingsHIV ~ vlSuppress
## ----------------------------------------------------
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## No    21 35.6 38 64.4    59  100    0.47648 0.53526
## Yes    6 46.2  7 53.8    13  100
## Total 27 37.5 45 62.5    72  100
## ====================================================
## (Labels:)
## (SiblingsHIV: HIV+ Siblings)
## (vlSuppress: Viral Suppression)
## ============================================================
```

```
## Tabulation: ARV ~ vlSuppress
## ____________________________________________________________
##                 0  (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## 3TC+AZT+EFV     1  16.7  5 83.3     6  100    0.15820 0.17838
## 3TC+AZT+NVP    12  35.3 22 64.7    34  100
## ABC+3CT+EFV     5  33.3 10 66.7    15  100
## ABC+3TC+ATV/r   3 100.0  0  0.0     3  100
## ABC+3TC+LPV/r   5  38.5  8 61.5    13  100
## TDF+FDC+ATV/r   1 100.0  0  0.0     1  100
## Total          27  37.5 45 62.5    72  100
## ============================================================
## (Labels:)
## (ARV: ARV regimens)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: ARVpi ~ vlSuppress
## ____________________________________________________
##          0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## Non-PI 18 32.7 37 67.3    55  100    0.13242 0.15867
## PI      9 52.9  8 47.1    17  100
## Total  27 37.5 45 62.5    72  100
## ====================================================
## (Labels:)
## (ARVpi: ART PI-NonPI)
## (vlSuppress: Viral Suppression)
## =====================================================
## Tabulation: timeART.grp ~ vlSuppress
## _____________________________________________________
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## 3-24M     2 16.7 10 83.3    12  100    0.07426 0.06992
## 25M-5Yr 16 34.8 30 65.2    46  100
## 6-8Yr     6 60.0  4 40.0    10  100
## 9+Yr      3 75.0  1 25.0     4  100
## Total    27 37.5 45 62.5    72  100
## =====================================================
## (Labels:)
## (timeART.grp: Time on ART Category)
## (vlSuppress: Viral Suppression)
## =====================================================
## Tabulation: ARVtab.grp ~ vlSuppress
## _____________________________________________________
##           0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## <=2.4    11 36.7 19 63.3    30  100    0.57680 0.58805
## 2.5-4.4   4 40.0  6 60.0    10  100
## 4.5-5.9   3 23.1 10 76.9    13  100
## 6+        9 47.4 10 52.6    19  100
## Total    27 37.5 45 62.5    72  100
## =====================================================
## (Labels:)
## (ARVtab.grp: number of ARV tablet/day)
## (vlSuppress: Viral Suppression)
## ====================================================
## Tabulation: cd4.grp ~ vlSuppress
## ____________________________________________________
```

```
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## <=350  5 71.4  2 28.6     7  100    0.05100 0.09470
## 351+  22 33.8 43 66.2    65  100
## Total 27 37.5 45 62.5    72  100
## ===================================================
## (Labels:)
## (cd4.grp: CD4 group: 350)
## (vlSuppress: Viral Suppression)
## ===================================================
## Tabulation: cd4.grp2 ~ vlSuppress
## ---------------------------------------------------
##        0 (r%)  1 (r%) Total (r%) Chi.Square F.Exact
## <=500  9 75.0  3 25.0    12  100    0.00329 0.00678
## 501+  18 30.0 42 70.0    60  100
## Total 27 37.5 45 62.5    72  100
## ===================================================
## (Labels:)
## (cd4.grp2: CD4 group: 500)
## (vlSuppress: Viral Suppression)
```

```r
## Summary measures Grouped by Viral Suppression
summBy(hiv.tables, vlSuppress)
```

```
## ==============================================================================
## Number Summary: Nregimens | timeART | vlSuppress
## by categories of: vlSuppress
## ------------------------------------------------------------------------------
##              Obs. NA. Mean Std.Dev Median  Q1  Q3 Normal.test  t-test Wilcoxon
## Nregimens.0    27   0  1.9     0.9      2   1   2     0.00017 0.01358  0.00358
## Nregimens.1    45   0  1.3     0.7      1   1   1     0.00000
## Nregimens.
## timeART.0      27   0  5.2     2.6    4.8 3.3 6.6     0.42649 0.03055  0.01458
## timeART.1      45   0  3.8     2.3    3.3 2.3 5.4     0.00122
## timeART.
## vlSuppress.0   27   0    0       0      0   0   0          NA      NA  0.00000
## vlSuppress.1   45   0    1       0      1   1   1          NA
## vlSuppress.
## ==============================================================================
## (labels: )
## (Nregimens: Previous Regimens)
## (timeART: Time on ART Years)
## (vlSuppress: Viral Suppression)
## (vlSuppress: Viral Suppression)
```

```r
## Logistic Regression: Sex
logit.sex <- glm(vlSuppress ~ Sex, data = hiv.tables, family = binomial)
fModelOutput(logit.sex)
```

```
## ==============================================
## Output of Logistic Regression
## Number of obs: 72
## ----------------------------------------------
##         + ------ uOR (95% CI) Pr(>|z|) ------ +
## SexMale |     0.9 (0.3 - 2.3)  0.76016        |
## ==============================================
## (Log-likelihood: -47.6)
```

```
## (AIC: 99.2)
```

```r
testModelFit(logit.sex)
```

```
##
##      Measure of model fit
##
## X-square = 0.09316955, df = 1, p-value = 0.7601858
##
## Interpretation:
##  The model as a whole does not fit significantly better than an empty model.
```

```r
## Logistic Regression: District
logit.district <- glm(vlSuppress ~ District, data = hiv.tables,
                      family = binomial)
fModelOutput(logit.district)
```

```
## ======================================================
## Output of Logistic Regression
## Number of obs: 72
## ------------------------------------------------------
##                + ------ uOR (95% CI) Pr(>|z|) ------ +
## Districtothers |    5.6 (1.4 - 38.1)  0.03098        |
## ======================================================
## (Log-likelihood: -44.5)
## (AIC: 93.0)
```

```r
testModelFit(logit.district)
```

```
##
##      Measure of model fit
##
## X-square = 6.22007, df = 1, p-value = 0.01263101
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

```r
## Logistic Regression: CD4
logit.cd4 <- glm(vlSuppress ~ cd4.grp2, data = hiv.tables,
                 family = binomial)
fModelOutput(logit.cd4)
```

```
## ====================================================
## Output of Logistic Regression
## Number of obs: 72
## ----------------------------------------------------
##              + ------ uOR (95% CI) Pr(>|z|) ------ +
## cd4.grp2501+ |    7.0 (1.8 - 34.4)  0.00717        |
## ====================================================
## (Log-likelihood: -43.4)
## (AIC: 90.8)
```

```r
testModelFit(logit.cd4)
```

```
##
##      Measure of model fit
##
## X-square = 8.465347, df = 1, p-value = 0.003619763
```

```
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

```
## Logistic Regression: Previous Regimens
logit.Nregimens <- glm(vlSuppress ~ Nregimens, data = hiv.tables,
                 family = binomial)
fModelOutput(logit.Nregimens)
```

```
## ==================================================
## Output of Logistic Regression
## Number of obs: 72
## --------------------------------------------------
##           + ------ uOR (95% CI) Pr(>|z|) ------ +
## Nregimens |     0.4 (0.2 - 0.8)  0.01347        |
## ==================================================
## (Log-likelihood: -44.1)
## (AIC: 92.2)
```

```
testModelFit(logit.Nregimens)
```

```
##
##      Measure of model fit
##
## X-square = 7.094486, df = 1, p-value = 0.007732145
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

```
## Logistic Regression: Time on ART
logit.timeART <- glm(vlSuppress ~ timeART, data = hiv.tables,
                 family = binomial)
fModelOutput(logit.timeART)
```

```
## ==================================================
## Output of Logistic Regression
## Number of obs: 72
## --------------------------------------------------
##         + ------ uOR (95% CI) Pr(>|z|) ------ +
## timeART |     0.8 (0.6 - 1.0)  0.03240        |
## ==================================================
## (Log-likelihood: -45.1)
## (AIC: 94.3)
```

```
testModelFit(logit.timeART)
```

```
##
##      Measure of model fit
##
## X-square = 5.00097, df = 1, p-value = 0.02533311
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

```
## Multivariable logistic Regression Model
logit.multi <- glm(vlSuppress ~ Sex + District + cd4.grp2,
                   data = hiv.tables, family = binomial)
```

```r
fModelOutput(logit.multi)
```

```
## ==================================================
## Output of unadjusted Logistic Regression
## Number of obs: 72
## __________________________________________________
##                + --- uOR (95% CI)  Pr(>|z|) --- +
## SexMale        |  0.9 (0.3 - 2.3)   0.76016   . |
## Districtothers | 5.6 (1.4 - 38.1)   0.03098   * |
## cd4.grp2501+   | 7.0 (1.8 - 34.4)   0.00717  ** |
## ==================================================
## ====================================================
## Output of Adjusted Logistic Regression
## Number of obs: 72
## ____________________________________________________
##                +   --- aOR (95% CI)  Pr(>|z|) --- +
## SexMale        |    0.9 (0.3 - 2.7)   0.83026   . |
## Districtothers | 17.7 (2.7 - 388.4)   0.01480   * |
## cd4.grp2501+   | 20.3 (3.3 - 401.5)   0.00685  ** |
## ====================================================
## (Log-likelihood: -38.1)
## (AIC: 84.2)
```

```r
testModelFit(logit.multi)
```

```
##
##      Measure of model fit
##
## X-square = 19.06844, df = 3, p-value = 0.0002646337
##
## Interpretation:
##  The model as a whole fits significantly better than an empty model.
```

```r
## Close the log file
ilog.close()
```

7 Miscellaneous functions

Functions not related to the domains or sets described above are demonstrated in this session.

7.1 Duplicate observations

The function `expand2` generates duplicated observations of a dataset based on the row indexes.

Let's try an example using `iris` dataset. We will duplicate the first 4 rows x 5 times.

```
# head iris
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```
# duplicate the first 4 rows.
iris_5 <- expand2(iris, 1:4, 5)
# check duplicated records
duplicates(iris_5)
```

```
## ====================================
## Duplicates in terms of all variables
## ------------------------------------
##   + Copies + Observations + Surplus +
## 1 |      1 |          144 |       0 |
## 2 |      2 |            2 |       1 |
## 3 |      3 |            0 |       0 |
## 4 |      4 |            0 |       0 |
## 5 |      5 |            0 |       0 |
## 6 |      6 |           24 |       4 |
## ====================================
## (Note:)
## (Total No. of obs: 170)
```

```
# we can just keep our duplicates, not all records
iris_5new <- expand2(iris, 1:4, 5, original = FALSE)
# check duplication
duplicates(iris_5new)
```

```
## ====================================
## Duplicates in terms of all variables
## ------------------------------------
##   + Copies + Observations + Surplus +
## 1 |      1 |            0 |       0 |
## 2 |      2 |            0 |       0 |
## 3 |      3 |            0 |       0 |
## 4 |      4 |            0 |       0 |
## 5 |      5 |           20 |       4 |
## ====================================
## (Note:)
## (Total No. of obs: 20)
```